
КОНВЕЙЕРЫ

Анализ задачи распределения ресурсов при помощи конвейеров и маршрутизаторов

Prishelets

2026

Введение

В данной работе рассматривается задача распределения ресурсов при помощи конвейеров и маршрутизаторов – в основном, на основе компьютерной игры Satisfactory. Рассматриваются типизация, реализация и взаимодействие друг с другом таких объектов, как: конвейерная лента, конвейерный соединитель, конвейерный разъединитель, и прочие их надстройки. Рассматривается как общее решение задачи распределения ресурсов, так и некоторые частные задачи.

В данной работе рассмотрено множество методов решения одной и той же задачи распределения ресурсов, проведён анализ эффективности каждого метода, и приведены рекомендации по выбору метода в зависимости от конкретной ситуации.

Так же проведено глубокое (на мой взгляд) исследование поведения вышеупомянутых конвейерных объектов, и открыты некоторые необычные установки, которых я ранее не встречал – они так же глубоко проанализированы с помощью инструмента математики

Никаких чрезмерно глубоких познаний в математике для понимания работы не требуется – все операции находятся в пределах школьной математики. Те вещи, что за школьную математику выходят – либо будут вкратце объяснены, либо понятны из контекста. Мною активно используется аппарат элементарной алгебры и кванторная нотация, также встречаются общая алгебра (теория групп) и линейная алгебра. ПРЕДПОЛОЖИТЕЛЬНО, по ходу дальнейшей разработки данной работы, также появятся: анализ функций одной и многих переменных, теории вероятностей, статистики, и дифференциальных уравнений.

К читателю: Это моя первая работа – так что не судите строго. любая конструктивная критика будет принята, проанализированна, и поможет улучшить имеющиеся наработки. За некоторое (довольно большое) число таких итераций, мы добьёмся качественой работы.

Contents

1 Терминология **4**
- 1.1 Конвейер . . . 4
- 1.2 Конвейерный разъединитель . . . 5
- 1.3 Конвейерный соединитель . . . 5
- 1.4 Узлы . . . 6

2 Быстрые системы **7**
- 2.1 Введение . . . 7
- 2.2 Теорема о числе перестановок . . . 8
- 2.3 Теорема о делимости . . . 9
- 2.4 Теорема об упрощении . . . 9
- 2.5 Алгебра задачи . . . 11
  - 2.5.1 Теория групп . . . 11
  - 2.5.2 Теория графов . . . 12
  - 2.5.3 Линейная алгебра . . . 13

3 Вычисления в быстрых системах **15**
- 3.1 Примеры . . . 15
  - 3.1.1 Упрощение . . . 15
  - 3.1.2 Рекурсия . . . 16
  - 3.1.3 Вариативность . . . 17
  - 3.1.4 Большие схемы . . . 18
  - 3.1.5 Несколько входов . . . 20
- 3.2 Решение . . . 21
- 3.3 Построение схемы . . . 24
  - 3.3.1 Степенное дерево . . . 24
  - 3.3.2 Ёлочка . . . 25
  - 3.3.3 Палочка . . . 27

4 Медленные системы **28**
- 4.1 Тупики . . . 28
- 4.2 Избыток на выход . . . 29
- 4.3 Теорема об инверсии . . . 32
- 4.4 Затворы . . . 34

5 Многоресурсные системы **36**
- 5.1 Вычитатель . . . 36
- 5.2 Вестгейт . . . 38

Chapter 1

Терминология

1.1 Конвейер

Конвейер – средство непрерывной перевозки ресурсов между двумя любыми точками.

Каждый конвейер характеризуется:

- Формой

  Описывается системой уравнений:

  $$\vec{r}(t) = \begin{cases} x_1 = x_1(t) \\ \vdots \\ x_n = x_n(t) \end{cases} \subseteq \mathbb{R}^n$$

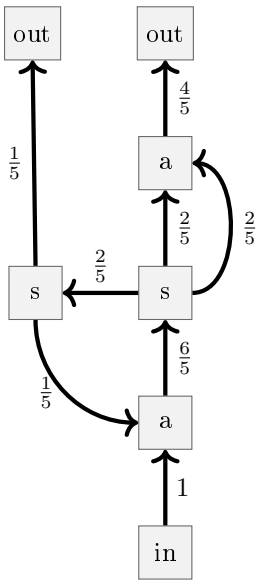
  В частности: $\mathbb{R}^3$ (Satisfactory) или $\mathbb{R}^2$ (Factoro)

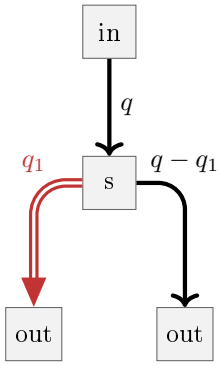
- Функцией распределения динамики скорости вдоль конвейера

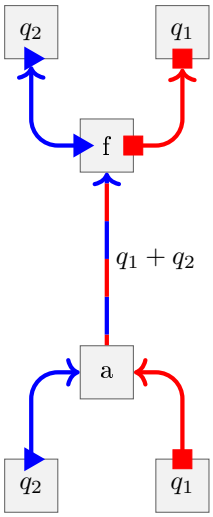
  $v(\vec{r}, t) \cong v(t_{\text{path}}, t_{\text{time}}) = v\left(\vec{t}\right)$

- Начальной $\vec{r}(0)$ и конечной $\vec{r}(t \to \max)$ точкой

На чертежах обозначается кривой, либо отрезком, соединяющим пару точек. Может иметь стрелки для указания направления движения, а рядом может подписываться пропускная способность. Цветом может обозначаться тип перевозимых ресурсов, а формой – пропускная способность







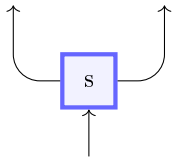
1.2 Конвейерный разъединитель

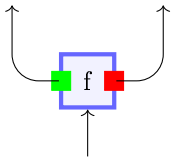
Конвейерный разъединитель (англ: splitter) – устройство, разъединяющее одну конвейерную ленту на несколько других.

Каждый конвейерный разъединитель характеризуется:

- числом выходных лент $n \in \mathbb{N}$
- правилами вывода: $\{\varphi_i\}$

На чертежах обозначается синей фигурой (обычно, квадрат). Буква внутри ("s" – splitter, "f" – filter, итд) характеризует подтип устройства. На выходах могут находиться специальные метки, обозначающие правила вывода φ_i. К разъединителю с одной стороны подводится конвейер, а со всех остальных – конвейеры отводятся.





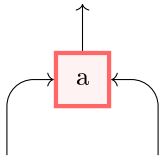
1.3 Конвейерный соединитель

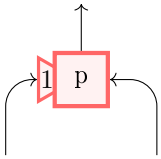
Конвейерный соединитель (англ: adder/merger) – устройство, соединяющее несколько входных лент в одну.

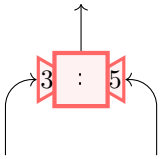
Каждый конвейерный соединитель характеризуется

- Числом входных лент $n \in \mathbb{N}$
- Правилами приёма(ввода): $\{\varphi_i\}$

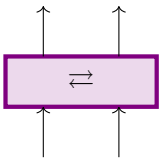
На чертежах обозначается красной фигурой (обычно, квадрат). Буква внутри ("a" – adder, "p" – priority, итд) характеризует подтип устройства. На входах могут находиться специальные метки, обозначающие правила ввода φ_i. От соединителя с одной стороны конвейер отводится, а к остальным – подводятся.

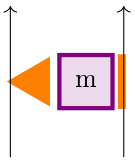






Также существуют специальные объекты, которые не являются ни соединителями ни разъединителями в прямом смысле: они как бы перемешивают ресурсы между конвейерами:





1.4 Узлы

Узлы (или узловые объекты) – это объекты, которые не производят ни соединение, ни разъединение конвейеров, ни даже просто взаимодействие параллельных конвейеров; но своим нахождением на них так или иначе меняют поведение системы.

Такими объектами могут служить, например:

- Ввод/вывод ресурсов
- Тупик
- Буфер
- Разрыв ленты
- ...



Chapter 2

Быстрые системы

2.1 Введение

Рассмотрим следующую систему:

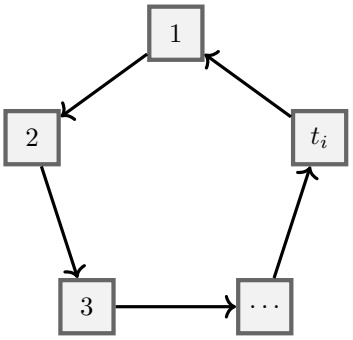
- Конвейеры имеют всюду равномерную скорость v, притом достаточно большую, чтобы можно было пренебречь взаимодействием между ресурсами, и преодолеваемыми ими расстояниями (а соответственно – и временем)

- Конвейерные разъединители работают по следующим правилам:

  – Всевозможные их числа выходов образуют множество $\mathbb{T} = \{t_1, ..., t_n\}$

  – Правило вывода: чередовать ресурс между выходами

  Схема очерёдности:



  – При этом распределение происходит мгновенно: как только ресурс заходит – так только и выходит

- Конвейерный соединитель может принимать любое число ресурсов, и мгновенно объединяет их в один конвейер (очерёдность не важна, т.к. ресурсы друг с другом не взаимодействуют)

В связи с мгновенностью распределений, назовём такую систему "быстрой".

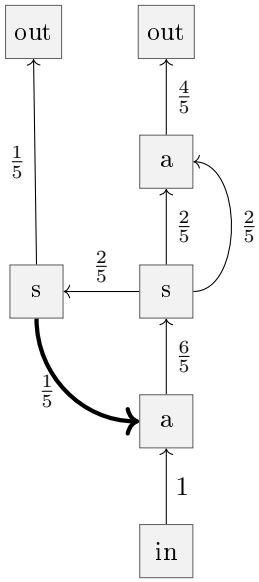
В быстрых системах определён следующий набор действий:

- Разделение (англ: splitting/chopping) – разделение одного конвейера на n равных частей

- Соединение (англ: merging) – соединение n конвейеров в один

- Возврат (англ: re-looping/re-cycling) – сначала отделение некоторой части потока, а затем соединение этой части с началом самого потока

  Пример:

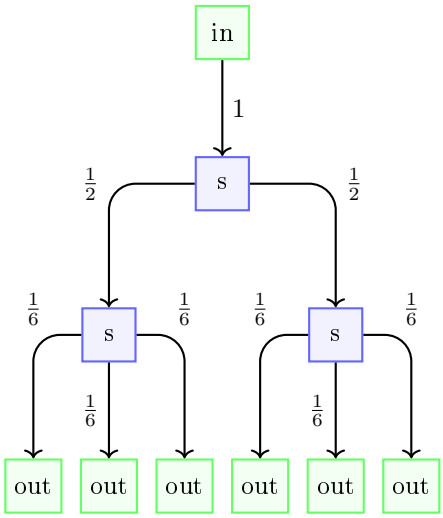


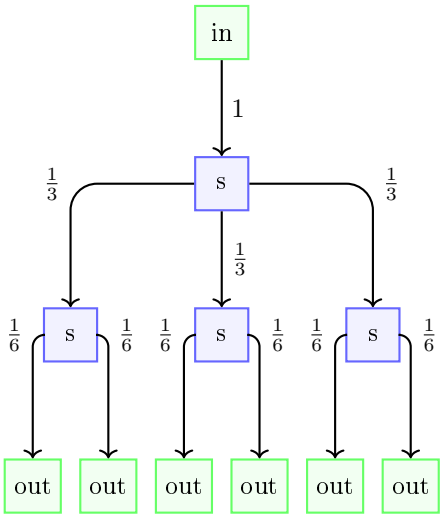
- Отделение[1] (англ: separating) – набор из последовательных и параллельных разделений, соединений и возвратов, с целью отделения заданной части q_1 ресурсов от изначальной конвейерной ленты, перевозящей q_0 ресурсов ($q_1 < q_0$)

Назовём все числа вида $s = \prod_{i=1}^{n} t_i^{a_i}$, где $a_i \in \mathbb{N}_0$ "стандартными". Тогда каждое число, обратное стандартному[2] может быть получено, как отделение от конвейера в 1 при помощи смеси t_1-арного, ... , и t_n-арного деревьев.

2.2 Теорема о числе перестановок

Для множества $\mathbb{T} = \{2, 3\}$, число $2^1 3^1 = 6$ является стандартным, тогда 1/6 является единичной дробью, её можно получить двумя способами:





[1] **Раз**деление и **от**деление отличаются тем, что **Раз**деление – это построение n-арного дерева; а **от**деление – это построение второй ветви, по которой едет заданное число ресурсов. Более заметно это различие в английской терминологии, а в русской различие заключается лишь приставкой...

[2] Будем называть такие числа единичными стандартными, или просто единичными. А если в числителе стоит не единица – то будем называть их стандартными долями.

Как можем заметить, дробь одна, а варианта её воспроизводства – два. Отсюда сформулируем теорему:

Любую единичную дробь, инициализированную числом $s = \prod_{i=1}^{n} t_i^{a_i}$ можно построить числом способов, равным числу перестановок его сомножителей:

$$\mathrm{P}\left[\sum_{i=1}^{n} a_i\right](a_1, a_2, \ldots, a_n) = \frac{\left[\sum\limits_{i=1}^{n} a_i\right]!}{\prod\limits_{i=1}^{n}(a_i!)}$$

2.3 Теорема о делимости

Если некоторую стандартную долю конвейера $\frac{a}{s}$ $(a < s)$ возвратить в его начало – то на участке до начала отделения пойдёт $q_0 + \frac{a}{s} \cdot q_0$, после второй рекурсии – $q_0 + \frac{a}{s} \cdot \left(q_0 + \frac{a}{s} \cdot q_0\right)$, и так далее.

Таким образом, в конечном итоге по данному участку будет идти:

$$\sum_{i=0}^{\infty} q_0 \cdot \left(\frac{a}{s}\right)^i = q_0 \cdot \frac{1}{1 - \frac{a}{s}} = q_0 \cdot \frac{s}{s - a}$$

Так как a можно выбрать произвольное – то от конвейера можно отделить даже не стандартную долю.

Обозначим операцию преобразования дроби $\frac{a}{s}$ в дробь $\frac{a}{b}$ $(b \leq s)$ за:

$$\frac{a}{s} - \frac{s-b}{s} := \frac{a}{s} \cdot \frac{1}{1 - \frac{s-b}{s}} = \frac{a}{b}$$
$$\frac{a}{s} - \frac{b}{s} := \frac{a}{s} \cdot \frac{1}{1 - \frac{b}{s}} = \frac{a}{s-b}$$

Теорема: любой единичный конвейер можно разделить на любые два положительных рациональных, в сумме дающих единицу, как рациональные доли входного конвейера при помощи операций отделения и возврата

$$\forall\ q_0, q_1, q_2 \in \mathbb{Q}_0^+\ (q_1 + q_2 = q_0)\ :$$
$$\exists\ a, b, s \in \mathbb{N}_0\ (a, b \leq s)\ (s - \text{standart})\ |$$
$$q_1 = \frac{a}{s} - \frac{b}{s}\ ,\ q_2 = \frac{s - a - b}{s} - \frac{b}{s}$$

2.4 Теорема об упрощении

Назовём системы:

- "простыми", если: $\forall\ t_i \in \mathbb{T}\ :\ t_i$ – простое
  Пример: $\mathbb{T} = \{2, 3\}$, $\mathbb{T} = \{11, 23, 41\}$, $\mathbb{T} \subseteq \{2, 3, 5, 7, 11, 13, \ldots\}$
- "полупростыми", если: $\forall t_i, t_j \in \mathbb{T}\ :\ \gcd(t_i;\ t_j) = 1$
  Пример: $\mathbb{T} = \{2, 3\}$, $\mathbb{T} = \{4, 9\}$, $\mathbb{T} = \{6, 7, 25\}$
- "дублирующими", если: $\exists\ t, a, b \in \mathbb{T}\ ;\ x \in \mathbb{N}_0\ ;\ y, z \in \mathbb{N}\ |\ t^z = a^x b^y$
  Пример: $\mathbb{T} = \{2, 3, 6\}$, $\mathbb{T} = \{2, 4\}$, $\mathbb{T} = \{4, 6, 9\}$

Теорема: любую дублирующую систему можно упростить:

$$\forall\ \mathbb{T} \mid \exists\ t, a, b \in \mathbb{T}\ ;\ x \in \mathbb{N}_0\ ;\ y, z \in \mathbb{N} \mid t^z = a^x b^y\ :$$
$$\exists\ \mathbb{T}' = \mathbb{T} \setminus \{t\ :\ \exists\ t, a, b \in \mathbb{T}\ ;\ x \in \mathbb{N}_0\ ;\ y, z \in \mathbb{N} \mid t^z = a^x b^y\} \neq \emptyset$$
$$\mathbb{T}' - \text{простое или полупростое}$$

Таким образом не имеет смысла отдельно изучать дублирующие системы, ведь для них будут верны все свойства полупростых (и простых соответственно). Однако сами по себе дублирующие системы могут быть удобны в решении отдельных задач: например, если в $t = \prod_i t_i^{a_i}$,
$\sum_i a_i$ – достаточно большое – то:

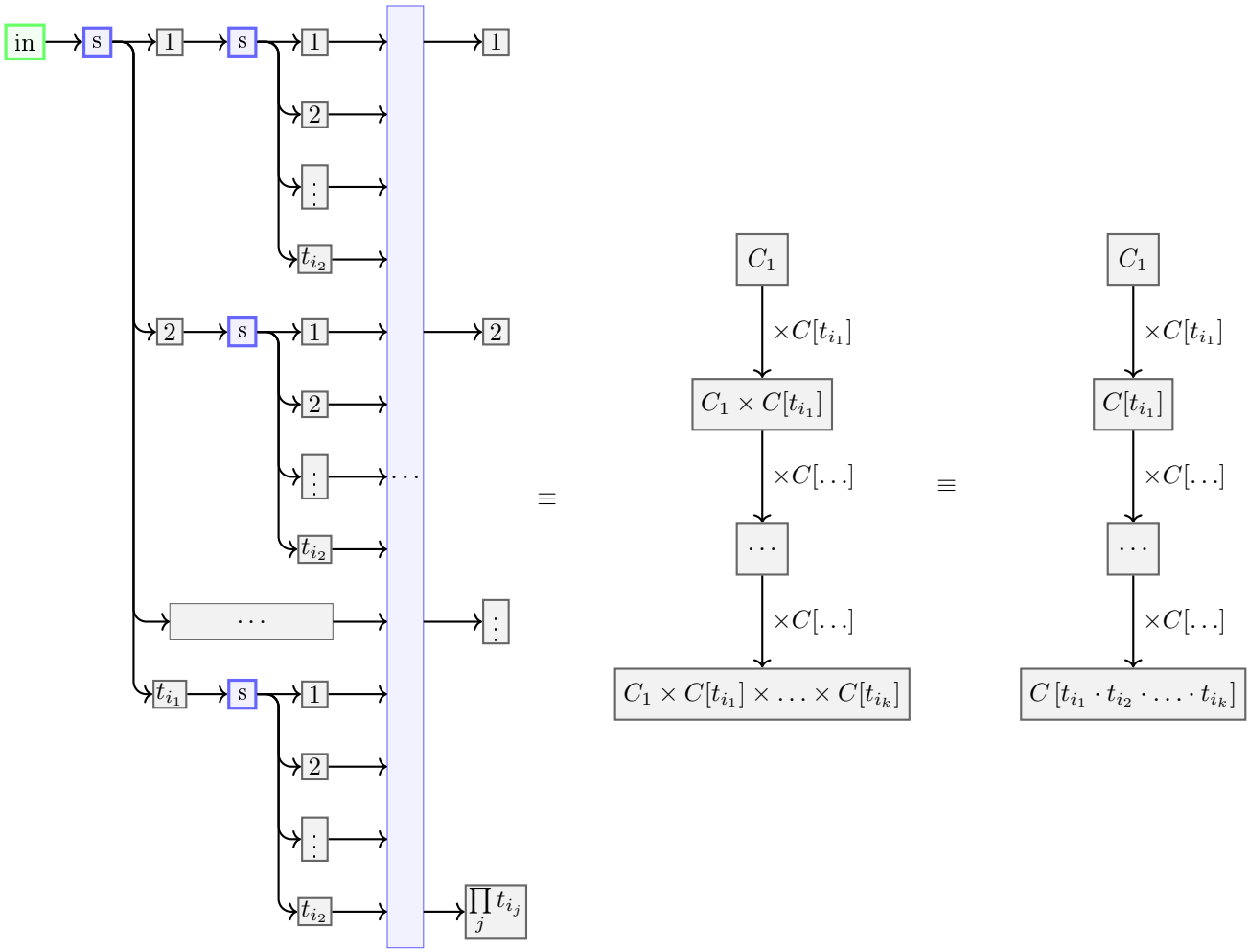
1. Система, реализующая единичную дробь, инициализированную данным числом, будет банально громоздкой
2. Возможных способов реализации такой системы тоже будет много, что значительно усложняет решение задачи

Тогда действительно проще установить готовый разветвитель, а не строить его с нуля.

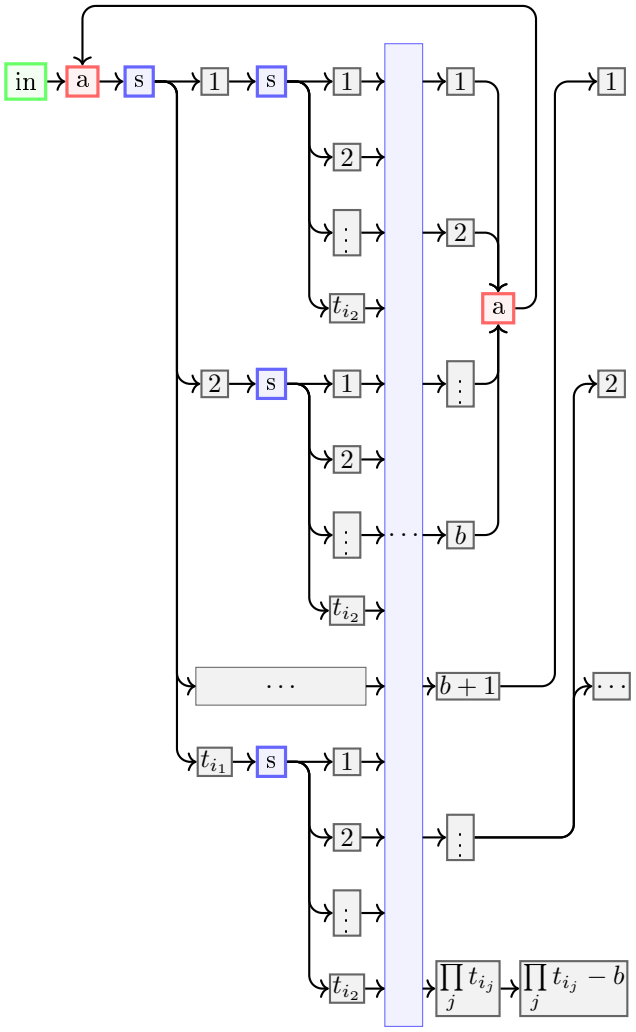
2.5 Алгебра задачи

2.5.1 Теория групп

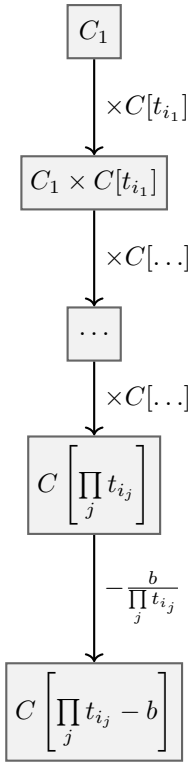
Так же хочется отметить, что каждый разделитель образует циклическую группу C_{t_i} – тогда резонно сказать, что конвейер в начале своего пути является циклической группой C_1, а каждый раз, применяя к нему операцию разделения при помощи серии параллельных сплиттеров, мы, по сути, домножаем группу конвейера на группу сплиттера:



Тогда всё множество выходных конвейеров образует циклическую группу $C\left[\prod_j t_{i_j}\right] = C\left[\prod_i t_i^{a_i}\right]$. Если из них принудительно b штук вернуть в начало – то мы снова придём к теореме о делимости, и получим уменьшение периода группы: $C[s] \xrightarrow{-b} C[s-b]$



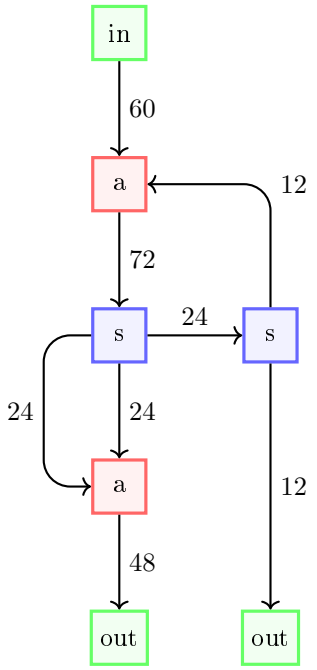
$\equiv$



Схемы, приведённые справа являются биективными для любой неупрощённой диаграммы разделения, но как мы увидим далее, многие, первоначально кажущиеся сложными и громоздкими схемы – можно довольно сильно сокращать, но тогда мы не сможем рисовать к ним данные схемы так просто. Тем более, такие схемы можно рисовать только для схем *разделения*, но не *отделения*.

2.5.2 Теория графов

Также хочется отметить, что, поскольку каждая схема является мультиорграфом – то её можно описывать не в виде чертежа, а, например, в виде матрицы смежности:



≡

\	a1	a2	s1	s2	out1	out2
in	1					
a1			1			
a2					1	
s1		2		1		
s2	1					1
in	60					
a1			72			
a2					48	
s1		24		24		
s2	12					12

Таблицу, удобства ради, давайте разделим на две части: сверху отметим обычную матрицу смежности для мультиорграфа: число в ячейке говорит о том, сколько рёбер идёт из одной вершины в другую. А снизу – пропускную способность одного ребра.

Такой граф состоит из одной компоненты слабой связности, а все её компоненты сильной связности – это области, благородя которым происходит операция возврата. Посмотрев на полустепень исхода каждого из узлов компоненты сильной связности, можно выяснить, какой коэффициент усиления она реализует.

Однако матричный метод не является удобным, если все элементы $\mathbb{T}$ – малы, а сам граф – большой. Эффективность он приобретает, когда число выходных лент большое, а кратных рёбер при этом мало.

2.5.3 Линейная алгебра

Рассмотрим задачу перераспределения ресурсов между n входами и k выходами: схему перераспределения можно представить в виде матрицы:

$$\begin{bmatrix} m_{11} & \dots & m_{1n} \\ \vdots & \ddots & \vdots \\ m_{k1} & \dots & m_{kn} \end{bmatrix} \times \begin{bmatrix} \text{in}_1 \\ \vdots \\ \text{in}_n \end{bmatrix} = \begin{bmatrix} \text{out}_1 \\ \vdots \\ \text{out}_k \end{bmatrix}$$

Рассмотрим матрицу $A := a_{ij} = \frac{\text{out}_i}{\text{in}_j}$. В ней можно выделить следующие классы значений:

- $a_{ij} = 1$
  В этом случае конвейер надо просто провести прямо, без промежуточных преобразований
- $a_{ij} < 1$
  В этом случае можно отделить соответствующую долю, а остаток станет новым входом, либо можно сохранить на следующие итерации.
- $a_{ij} > 1$
  В таком случае итоговый конвейер получится, как сложение данного с некоторыми другими (или преобразования над ними)

Если в такой матрице находится единичный элемент – то в оригинальной матрице на этом же месте мы ставим 1, а остальные значений в тех же столбце и строке – заполняем нулями, а в первоначальной матрице

они удаляются. Среди оставшихся элементов можно выбрать "диагональный" набор дольных значений, и по тем же принципам их занести в итоговую матрицу. Таким образом, должен остаться лишь набор элементов $a_{ij} > 1$. Тогда нам остаётся лишь честно искать решения.

Помочь в этом может следующая теорема: сумма значений в столбцах матрицы преобразования должна давать 1:

$$\begin{bmatrix} m_{11} & \dots & m_{1n} \\ \vdots & \ddots & \vdots \\ m_{k1} & \dots & m_{kn} \end{bmatrix} \times \begin{bmatrix} \text{in}_1 \\ \vdots \\ \text{in}_n \end{bmatrix} = \begin{bmatrix} \text{out}_1 \\ \vdots \\ \text{out}_k \end{bmatrix}$$

$$\sum_{i=1}^{k} m_{ji} = 1$$

Таким образом, нам остаётся, всего-то навсего, решить систему $n + k$ уравнений и nk переменных:

$$\begin{cases} m_{11} \cdot \text{in}_1 + \ldots + m_{1n} \cdot \text{in}_n = \text{out}_1 \\ \vdots \\ m_{k1} \cdot \text{in}_1 + \ldots + m_{kn} \cdot \text{in}_n = \text{out}_k \\ m_{11} + \ldots + m_{k1} = 1 \\ \vdots \\ m_{1n} + \ldots + m_{kn} = 1 \end{cases}.$$

Как известно, такая система имеет единственное решение только если $nk = n + k$, что имеет только две пары целочисленных решений: $(0, 0)$ и $(2, 2)$. Если же точка находится где-то внутри – то решение тривиально: соединение конвейеров, либо обычное отделение. В противном же случае – решений бесконечно много. Если выходов больше, чем входов – то входы можно банально поделить при помощи имеющихся у нас разделителей, и тогда, в какой-то момент мы сможем отделить все требуемые числа. Если входов больше, чем выходов – то наоборот, надо соединять.

Chapter 3

Вычисления в быстрых системах

3.1 Примеры

Рассмотрим $\mathbb{T} = \{2, 3\}$

3.1.1 Упрощение

От конвейера в 60 требуется отделить 15 и 45:

$\frac{15}{60} = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2}$, $\frac{45}{60} = \frac{3}{4} = 3 \cdot \frac{1}{2} \cdot \frac{1}{2}$

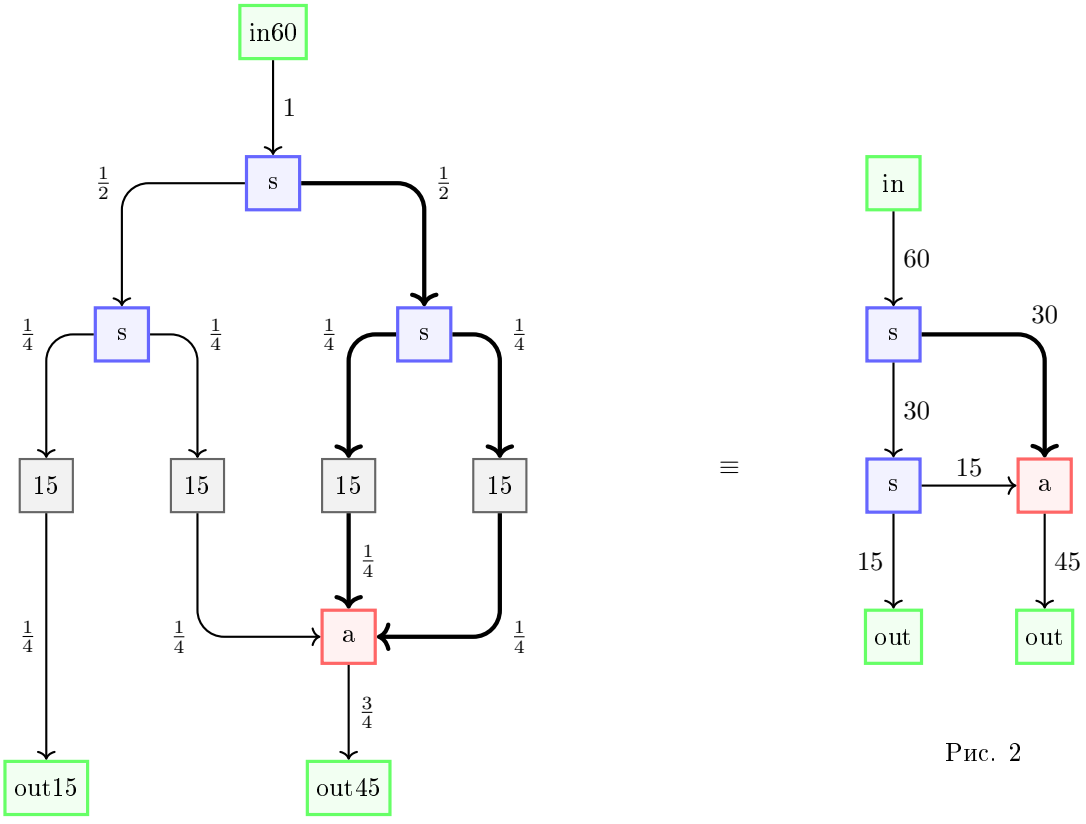
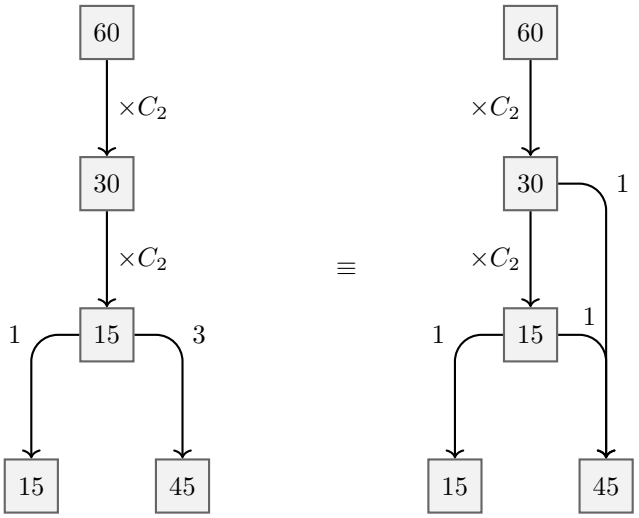


Рис. 1

Рис. 2

Заметим, что на рис. 1 имеется **область**, в которой разделение бессмысленно, ведь отделённые друг от друга конвейеры затем вновь соединяются. Тогда можно убрать данный разделитель, оставив целиковый **конвейер**.

Для рассмотрения алгебраической картины данной задачи, нам потребуется ввести обозначение: за ответвления в стороны, с припиской в виде числа будем обозначать, что столько-то конвейеров уходят в новую ветку схемы:

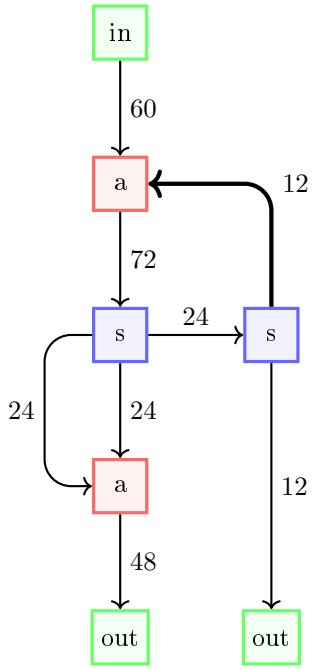


Как видим, при упрощении схемы мы получим, что число конвейеров, идущих на соединение не превышает период циклической группы последнего сплиттера: это будет важным моментом, когда мы будем писать программное решение к задаче.

3.1.2 Рекурсия

От конвейера в 60 требуется отделить 12

$\frac{12}{60} = \frac{1}{5} = \frac{1}{6} - \frac{1}{6} = \frac{1}{2} \cdot \frac{1}{3} - \frac{1}{2} \cdot \frac{1}{3}$, $\frac{60-12}{60} = \frac{48}{60} = \frac{4}{5} = \frac{4}{6} - \frac{1}{6} = \frac{2}{3} - \frac{1}{2} \cdot \frac{1}{3}$

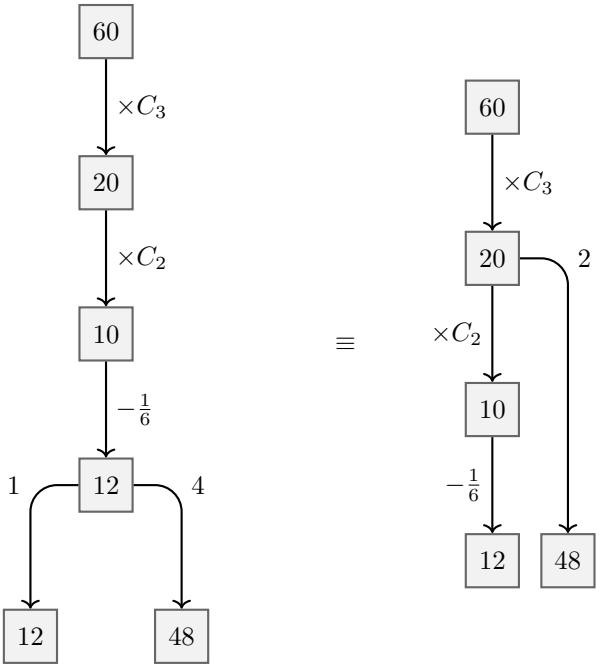


Пускай, в систему импульсами подаётся по одному ресурсу раз в некоторое время[1], тогда внутри делителя на 6 у каждого ресурса будет 6 различных выходов. Положим, первые 5 ресурсов ушли на свои 5 выходов, и как-то распределились в дальнейшей части конструкции. Тогда они поменяли данные, хранящиеся в памяти делителей таким образом, что 6-ой ресурс может уйти только на 6-ой выход. Но этот выход возвращён в начало

[1] Такая аппроксимация возможна в связи с тем, что конвейер имеет большую скорость, и ресурсы внутри системы друг с другом не взаимодействуют

системы, и тогда 6-ой ресурс становится 1-ым, так как все настройки делителей сбросились[2]. Таким образом, данный ресурс снова уходит на один из 5-ти допустимых выходов. Притом получается так, что порядок выходов одинаков, так что данная система является равносильной делителю на 5.

Рассмотрим же теперь алгебраическую картину данной задачи:



Забавно, что граф может разбиваться на несколько, казалось бы независимых ветвей, но применив операцию возврата в одной ветви – автоматически меняется значение и во всех других ветвях.

3.1.3 Вариативность

От конвейера в 60 требуется отделить 6. Рассмотрим все (сокращённые) варианты такой схемы:
$\frac{6}{60} = \frac{1}{10} = \frac{1}{12} - \frac{2}{12} = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{3} - \frac{1}{6}$

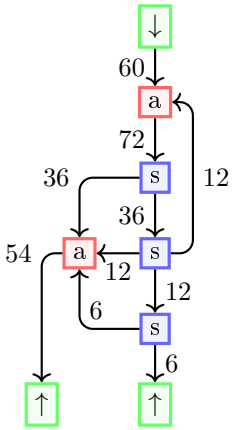


Рис. 1

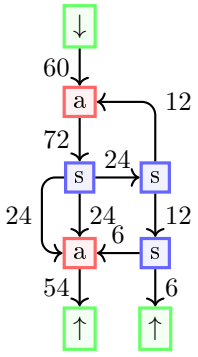


Рис. 2

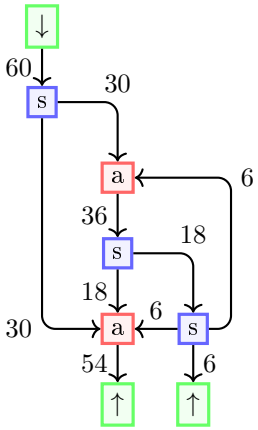


Рис. 3

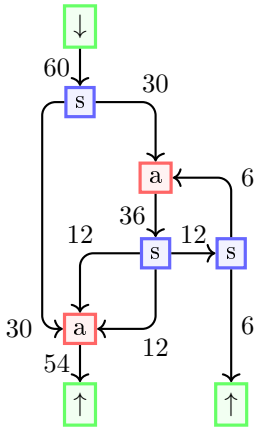
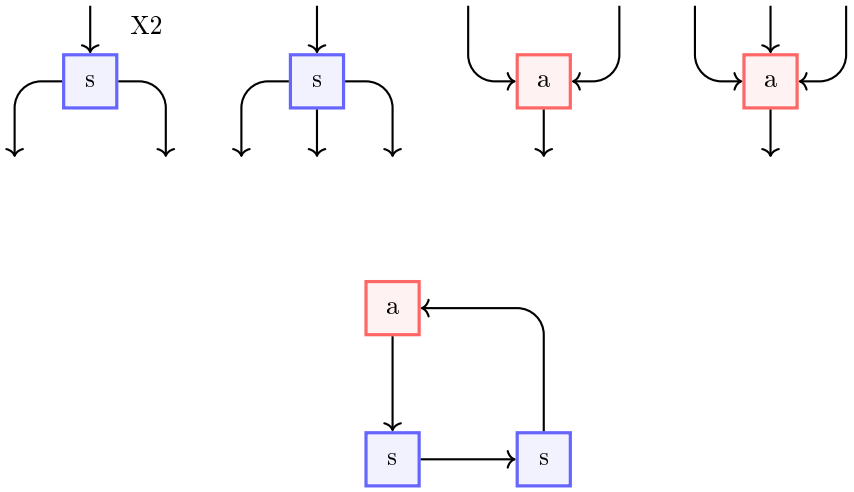


Рис. 4

[2]Данная система является примером группы $C_2 \times C_3 = C_6$, в которой 6-ой элемент является и 0-ым

Заметим, что в во всех схемах, являющихся сокращёнными вариациями реализации одного и того же решения, число соединителей и разъединителей – инвариантно относительно топологии графа. Притом сохраняется ещё и число входов/выходов конкретного узла, а так же сохраняются подструктуры по типу циклов:



Так же важно помнить, что это всё было только одно решение: $s = 12$, а ведь можно было взять первые пару ближайших стандартных чисел $(12, 16, 18, \ldots)$, или стандартные числа, являющиеся чисто степенями $(16, 32, 27, \ldots)$. Так же в решении можно было использовать нетривиальные разложения по типу: $\frac{5}{6} = \frac{1}{2} + \frac{1}{3}$

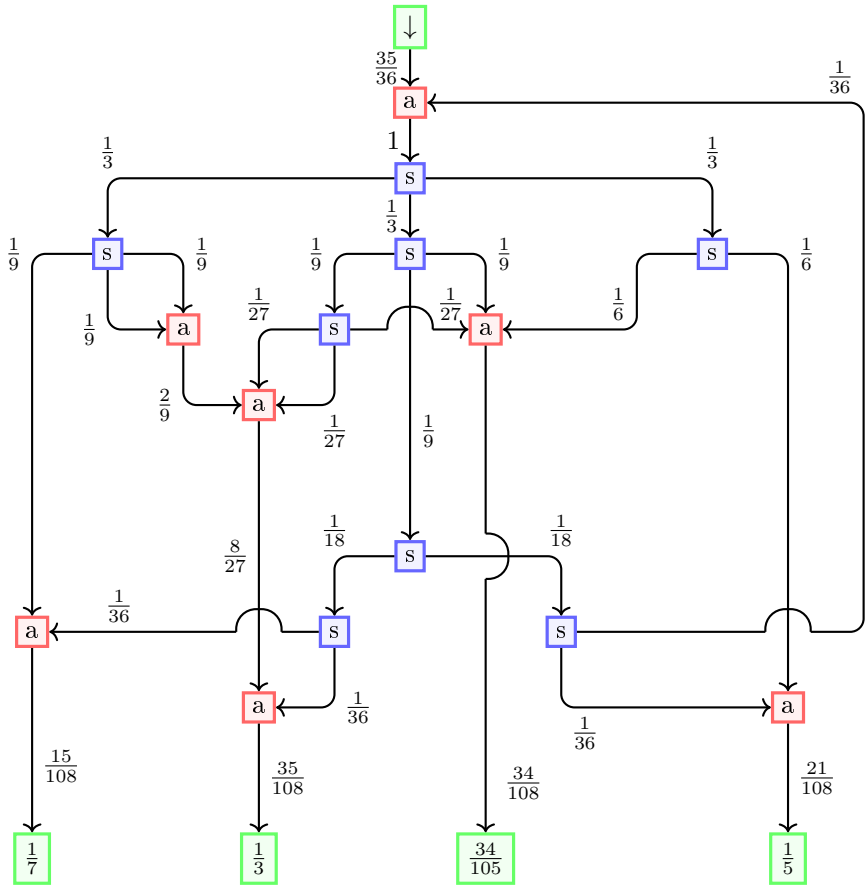
3.1.4 Большие схемы

Рассмотрим схему побольше: отделить $\frac{1}{7}$, $\frac{1}{5}$ и $\frac{1}{3}$ одновременно от одного конвейера.

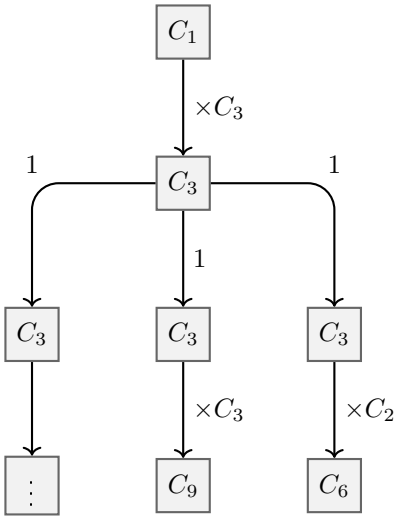
$\mathrm{lcm}(3, 5, 7) = 105$

$$\frac{1}{7} = \frac{15}{105} = \frac{15}{2^2 3^3} - \frac{3}{108} = \frac{5}{36} - \frac{1}{36} = \frac{1}{36} + \frac{1}{9} - \frac{1}{36}$$
$$\frac{1}{5} = \frac{21}{105} = \frac{21}{2^2 3^3} - \frac{3}{108} = \frac{7}{36} - \frac{1}{36} = \frac{1}{36} + \frac{1}{6} - \frac{1}{36}$$
$$\frac{1}{3} = \frac{35}{105} = \frac{35}{2^2 3^3} - \frac{3}{108} = \frac{1}{36} + \frac{2}{27} + \frac{2}{9} - \frac{1}{36}$$

Остаток: $\frac{34}{105}$



Как мы можем заметить, на схемах побольше появляются две интересные вещи: пересечеиния конвейеров[3] и разная периодичность параллельных ветвей:



Как мы видим, циклические группы C_9 и C_6 находятся в дереве деления на одинаковых уровнях. Это и есть пример применения нетривиальных разложений. Давайте же тогда рассмотрим некоторые разбиения, например, дроби $\frac{34}{108}$:

[3] Смотреть приложение **??**: проблема перекрёстков (brick factory problem)

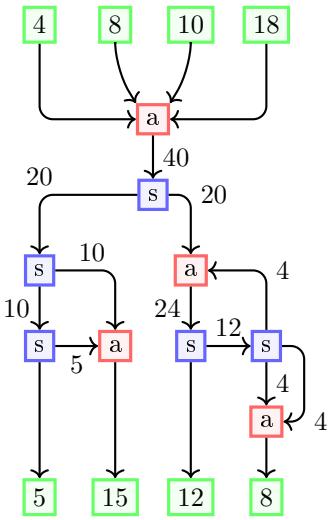
$$\frac{34}{108} = \frac{17}{54} = \frac{1}{54} + \frac{8}{27} = \frac{1}{54} + \frac{2}{27} + \frac{2}{9}$$
$$\frac{1}{27} + \frac{5}{18} = \frac{1}{27} + \frac{1}{18} + \frac{2}{9}$$
$$\frac{1}{27} + \frac{1}{9} + \frac{1}{6}$$

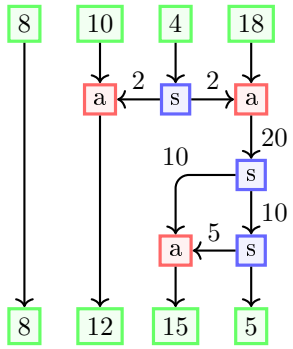
3.1.5 Несколько входов

Порой бывает такое, что мы имеем несколько входных конвейеров, идущих, например, с разных производств, и нам необходимо как-то перераспределить между ними ресурсы.

Можно пойти двумя путями: по принципу чайника[4] : соединить все входные конвейеры, и работать с тем, что получим; или поиграться с отделениями в самих конвейерах. Рассматривать мы будем второй метод, так как он является обобщением первого.

Пример:
Входы: 4, 8, 10, 18
Выходы: 5, 8, 12, 15





Очевидно, что вторая схема проще первой. Хотя, на самом деле, это не всегда так, и нам просто повезло. Иногда эти две схемы могут быть вообще одинаковыми (например от (10, 10, 10, 10) отделить (5, 8, 12, 15)).

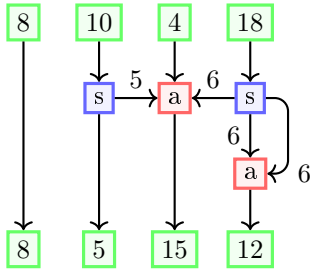
Однако даже так, её можно упростить: как нам уже известно, такая схема является матрицей преобразования вида:

$$\begin{bmatrix} 5 \\ 8 \\ 15 \\ 12 \end{bmatrix} = \begin{bmatrix} m_{11} & \dots & m_{14} \\ \vdots & \ddots & \vdots \\ m_{41} & \dots & m_{44} \end{bmatrix} \times \begin{bmatrix} 10 \\ 8 \\ 4 \\ 18 \end{bmatrix}$$

[4] Как физик заваривает чай, если чайник пустой? Наливает воду, кипятит воду, заваривает чай.
Как математик заваривает чай, если чайник пустой? Наливает воду, кипятит воду, заваривает чай.
Как физик заваривает чай, если в чайнике есть вода? Кипятит воду, заваривает чай.
Как математик заваривает чай, если в чайнике есть вода? Выливает воду, наливает воду, кипятит воду, заваривает чай.
Принцип чайника: сведи задачу к предыдущей.

И имеет решение, например:

$$\begin{bmatrix} 5 \\ 8 \\ 15 \\ 12 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \frac{1}{2} & 0 & 1 & \frac{1}{3} \\ 0 & 0 & 0 & \frac{2}{3} \end{bmatrix} \times \begin{bmatrix} 10 \\ 8 \\ 4 \\ 18 \end{bmatrix}$$



Таким образом, мы получили самую простую версию схемы перераспределения ресурсов между несколькими входами и выходами.

3.2 Решение

Любую стандартную долю можно представить, как сумму некоторого количества других стандартных дробей:

$$\forall \frac{a}{s} \ : \ \exists \ n \in \mathbb{N}, A = \{a_i\}, S = \{s_i\} \mid \frac{a}{s} = \sum_{i=1}^{n} \frac{a_i}{s_i}$$

"Элементарными" назовём несократимые стандартные дроби, для которых $n = 1$, т.е.:
$\nexists \ a_i \leq a \mid \gcd(a_i, s) \neq 1$

Для дроби $\frac{a}{b}$ "решением" назовём[5] $A_b^a = O_b^a - R_b$, где:

- $s = s(b)$ – ближайшее большее стандартное к b
- $O_b^a = O_s^a = \sum_{i=1}^{n} \frac{a_i}{s_i}$
- $R_b = O_s^{s-b}$

Тогда задача сводится к изучению получения оптимальных $S(b)$, O_s^a и R_b.

Код на C# для реализации поиска наименьшего $s(b)$ для множества $\mathbb{T} = \{2, 3\}$:

[5] под "-" подразумеваем его определение через рекурсию

C#

```
public static int hig_closest(int number, int x = 0, int y = 0, int branch = 1)
{
  int answer = -1;
  int num = Convert.ToInt32(Math.Pow(2, x) * Math.Pow(3, y));

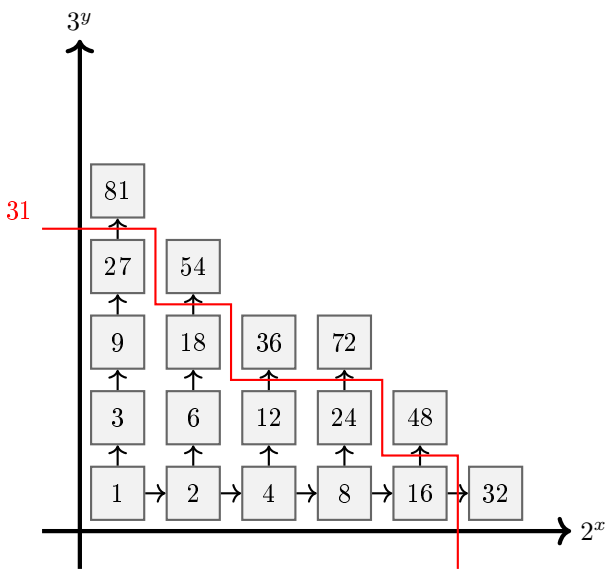
  if (num >= number) { answer = num; }
  else
  {
    List<int> answers = new List<int>();

    answers.Add(hig_closest(number, x + 1, y, 0));
    if (branch == 1)
    { answers.Add(hig_closest(number, x, y + 1, 1)); }

    answer = answers.Min();
  }

  return answer;
}
```

Он реализует следующее дерево поиска (пример для $b = 31$):



Нетрудно модифицировать данный алгоритм для произвольных $\mathbb{T}$:

Python

```
def hig_closest(target, powers, branch=max(T)):
  answer = -1
  num = 1
  for i in range(0, T.len()):
    num *= T[i]**powers[i]

  if num >= target: return num

  answers = []
  for i in T:
    if i <= branch>:
      answers.append(hig_closest(target, powers.increace(i), i))

  answer = min(answers)

  return answer
```

Вот нашли мы $s(b)$, а как теперь искать O_s^a? Ответ прост: брутфорс. Требуется перебрать все возможные способы разбиения числа a на суммы всевозможных натуральных[6] чисел, после чего соответствующие дроби (если это возможно) мы сокрращаем дроби, а затем объединяем все дроби с одинаковыми знаменателями. Для каждого отдельного знаменателя $s_i = \prod_{j=0}^{n} t_j^{a_j}$ должно быть выполненно: $s_i < t_j \ \forall j$

Алгоритм для поиска одного варианта разбиения:

Python

```python
def spliter(numerator, denominator):
  s = hig_closest(denominator)
  reduce_coef = gcd(s, numerator)
  numerator //= reduce_coef
  denominator = s // reduce_coef

  s = low_closest(numerator, denominator)

  if s == numerator:
    return [[numerator, denominator]]

  else:
    cdg1 = gcd(numerator-s, denominator)
    gcd2 = gcd(s, denominator)
    d_parts = [
      [(numerator-s)//gcd1, denominator//gcd1],
      [s//gcd2, denominator//gdc2]
    ]

  while 1:
    memory = []

    for fractions in d_parts:
      x, y = fractions[0], fractions[1]
      s = low_closest(x, y)

      if p != x:
        gcd1 = gcd(x-s, y)
        gcd2 = gcd(s, y)
        memory.append([(x-s)//gcd1, y//gcd1])
        memory.append([s//gcd2, y//gcd2])

      else: memory.append(fractions)

    if len(d_parts) == len(memory): break
    else: d_parts = memory[0::]

  return d_parts[0::]
```

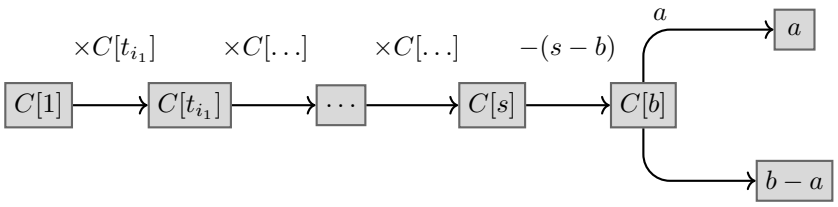
Полноценный алгоритм на C# приведён в satisfactory_conveyor_calc.sln

[6] Вцелом, можно и не натуральных, а просто рациональных: $\frac{2}{3} = \frac{1.5}{3} + \frac{0.5}{3} = \frac{1}{2} + \frac{1}{6}$ – такое разбиение вполне подойдёт для $\mathbb{T} = \{2, 6\}$. Однако, такой алгоритм слишком сложен, ведь перебирать придётся **значительно** больше вариантов

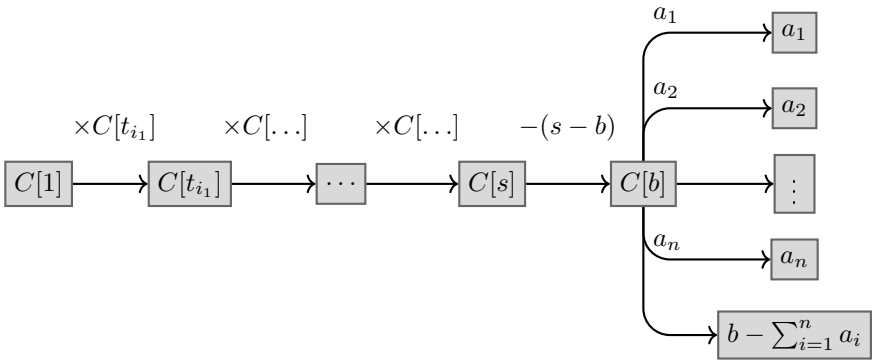
3.3 Построение схемы

3.3.1 Степенное дерево

Самый простой способ: если известно решение $A_b^a = O_b^a - R_b$ – то можно просто в лоб реализовать дерево на s листьев, $s-b$ из них вернуть в начало, a соединить, и отправить на выход; а оставшиеся $b-a$ тоже соединить, и тоже на выход.



Аналогично можно реализовать схему для отделения a_1, a_2, ..., a_n от b:



Однако, если $s = \prod_{i=1}^{k} t_{j_i}^{\alpha_i}$ – то такое решение займёт:

$$\left(k\ ;\ \prod_{i=1}^{k} t_{j_i}^{\alpha_i}\right)$$

Если требуется показать суть зависимости – то масштабы можно редуцировать до:

$$(x; e^x)$$

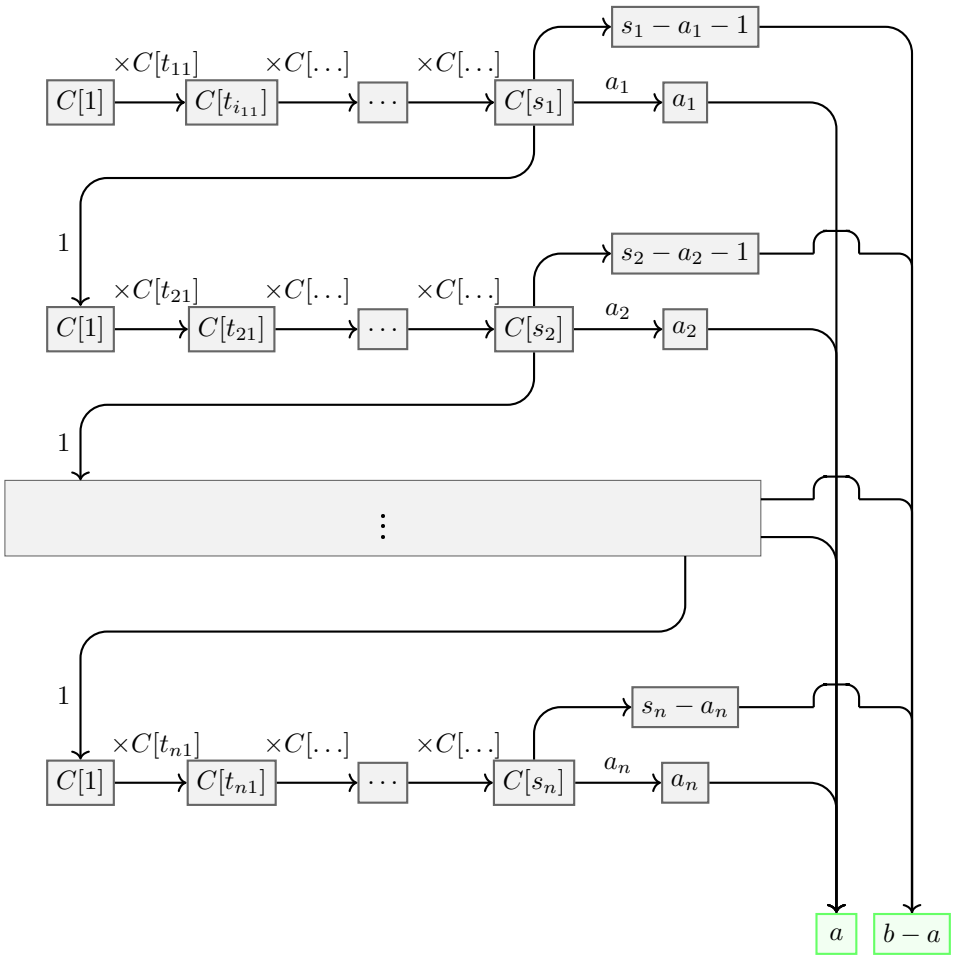
В общем: площадь растёт экспоненциально в зависимости от сложности схемы, что очень неудобно, особенно для сложных базисов.

3.3.2 Ёлочка

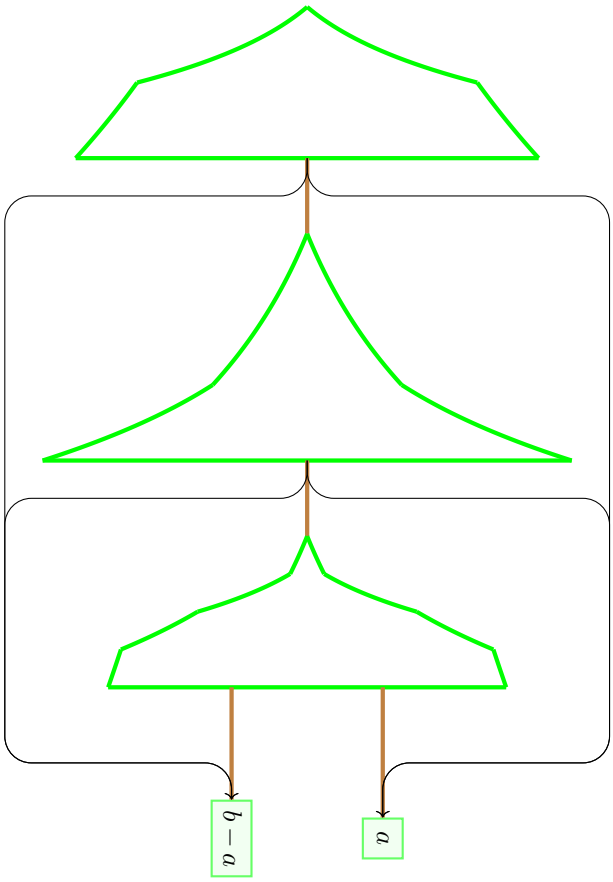
Однако, как мы уже знаем, схемы предыдущего вида можно сократить. первым уровнем аппроксимации упрощения является "ёлочка". Рассмотрим ситуацию $S(b) = b$:

$$\frac{a}{b} = \sum_{i=1}^{n} \frac{a_i}{s_i} \qquad s_i = s_{i-1} \cdot \prod_{j=1}^{k_i} t_{ij}^{\alpha_{ij}} \qquad s_0 = 1 \qquad a_i < \min(t_{ij})$$

Пускай, каждый t_i повторяется ровно α_i раз внутри одного s_i. Будем реализовывать s_i один за другим:



Казалось бы, почему "ёлочка", если это "змейка" какая-то? Дело в том, что следует рассмотреть граф, а не алгебраическую схему. Рассмотрим же возможный вид контуров графа:



И действительно: теперь проглядываются "ствол", и крона, которая сначала нарастает-нарастает-нарастает, а затем резко убывает в ноль, и оголяет ствол. И картинка становится похожа на ёлку.

Так же, такой тип схем занимает меньше места, чем простое дерево:

$$\sum_{i=1}^{n}\left(k_i\ ;\ \prod_{j=1}^{k_i} t_{ij}^{\alpha_{ij}}\right)$$

Или, если очень грубо округлять:

$$\left(k\ ;\ \max\left(\prod_{j=1}^{k_i} t_{ij}^{\alpha_{ij}}\right)\right)$$

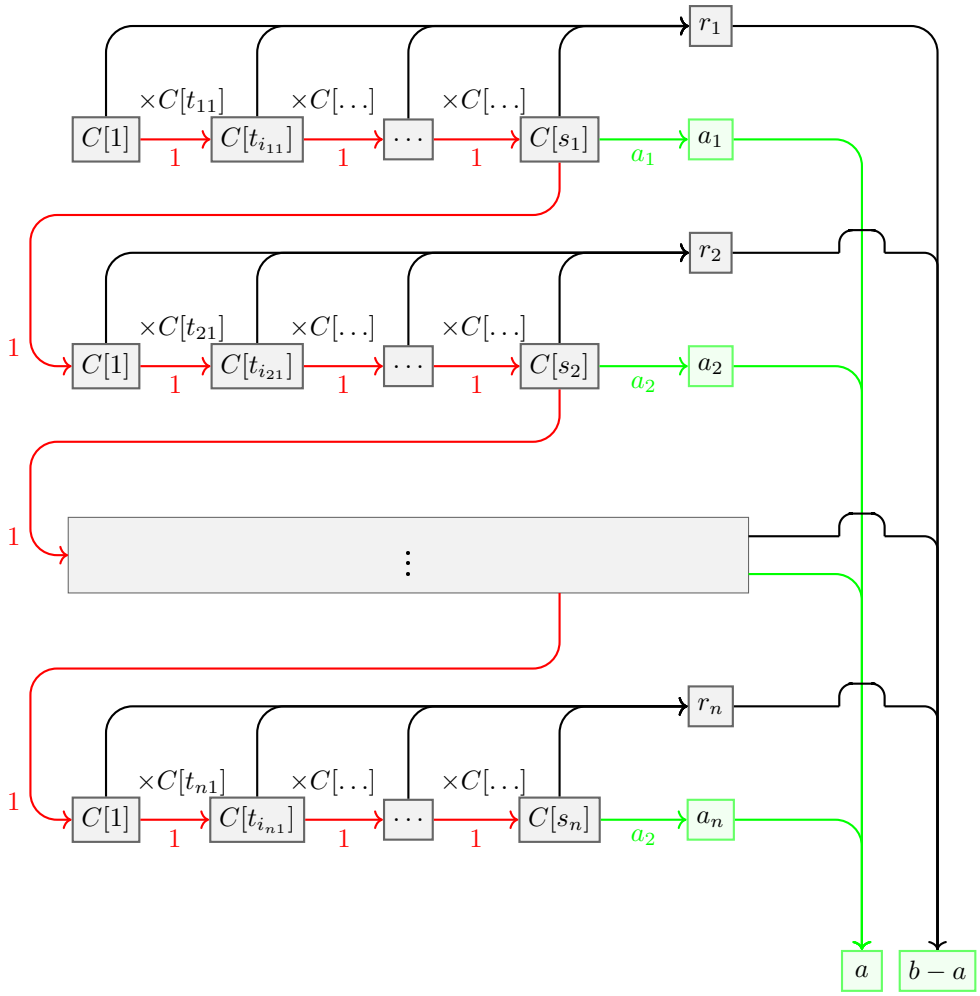
Работает это за счёт того, что мы поочерёдно реализуем небольшие степенные деревья, и предварительно успеваем их обрезать, предотвращая экспоненциальный взрыв. Суть та же, что и у линзы френеля. Таким образом, редуцировать размер можно до:

$$\left(x, e^{x/k}\right)$$

Так же, такую схему очень удобно реализовывать в 3D играх: каждую отдельную крону можно выносить на отдельный этаж, что позволит сильно сэкономить вплощади ценой высоты. Однако нарисовать общую схему под произвольное число выходов будет сложно, хотя реализовывать отдельно ничто не запрещает.

3.3.3 Палочка

При всём при этом, можно сократить даже ёлочку: раз у нас $a_i < \min(t_{ij})$ – то никто нам не гарантирует, что это верно идля остатка – так что остаток мы и будем скращать. Выберем любое $t_{ij} \mid a_i < t_{ij}$, и поставим его в конец очереди для s_i:



Забавно, что площадь схем уменьшается, то есть, казалось бы, схемы – упрощаются – а наше алгебраическое отображение только усложняется... Всё по тому, что увеличивается концентрация решений, а стало быть – и сложность. Собственно, площадь, занимаемая палочными схемами элементарна:

$$(k\ ;\ 2)$$

Вау! Линейная площадь! А была ведь экспоненциальная! В чём же подвох? А подвох в том, что данная схема абсолютно не масштабируема. Такие схемы можно реализовывать **только** для отделения одной величины в случае $S(b) = b$. Если $S(b) \neq b$ – То палочковую схему построить невозможно... По крайней мере в том виде, что подан в схеме: если повезёт – придётся избирательно выбирать порядок сплиттеров, а затем избирательно от них забирать конвейеры для возврата. А возможно придётся даже вести несколько параллельных линий, возможно даже с разными порядками действий, или вообще разными сплиттерами вприципе. Если же требуется отделить несколько значений – то предсказать уже невозможно ничего.

Chapter 4

Медленные системы

Возьмём известную нам "быструю систему", и добавим ограничение на скорость конвейера: введём множество $\mathbb{V}$ допустимых скоростей v_i, каждая из которых является вполне конечной. Например, в Satisfactory: $\mathbb{V} = \{60, 120, 270, 480, 720, 1200\}$. Тогда необходимо помимо абстрактной площади начать учитывать и длину конвейеров. А так же необходимо начать учитывать взаимодействие ресурсов и время этого взаимодействия.

Обычно, в играх по типу Satisfactory и Factorio "скорость" конвейера измеряется не в метрических единицах $[v_i] = $ м/с, а как пропускная способность $[q_i] = $ шт/с. Но одно можно перевести в другое, если добавить плотность ёмкости конвейера $[\rho_i] = $ шт/м. $v_i \cdot \rho_i = q_i$

Однако это всё не меняет допустимый набор действий, а просто накладывает на него некоторые ограничения, и меняет суть задачи: мы перераспределяем не просто ресурсы, а пропускные способности.

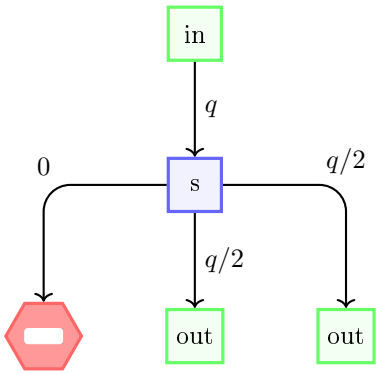
$$\begin{bmatrix} m_{11} & \dots & m_{1n} \\ \vdots & \ddots & \vdots \\ m_{k1} & \dots & m_{kn} \end{bmatrix} \times \begin{bmatrix} q_1^{\text{in}} \\ \vdots \\ q_n^{\text{in}} \end{bmatrix} = \begin{bmatrix} q_1^{\text{out}} \\ \vdots \\ q_n^{\text{out}} \end{bmatrix}$$

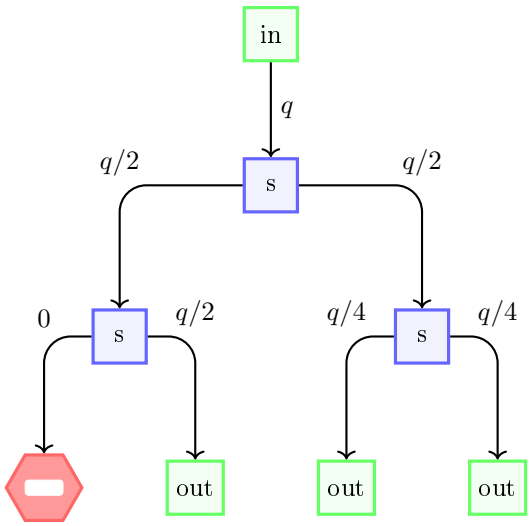
В матричной форме преобразования, элементы самой матрицы – скаляры, а элементы векторов – пропускные способности $[q_i] = $ шт/с

Важно отметить, что пропускная способность конвейера q_i и пропускная способность русурсов q^{res} – это две разные, но взаимосвязные вещи: q^{res} – это то, что производят заводы, и что "выливается" на конвейеры, которые двигаются с v_i, и имеют ρ_i. Для адекватной работы такой системы необходимо, чтобы $q^{\text{res}} \leq q_i$. В противном случае завод не сможет выкладывать ресурсы на конвейер – то есть будет простаивать. Таким образом, схемы необходимо проектироавть с учётом этих двух величин.

Если же $q^{\text{res}} > q_i$ – то возможны 2 принципиальные ситуации: $q_i = 0$ и $q_i \neq 0$. Рассмотрим первую на двух примерах:

4.1 Тупики





Если все конвейеры (кроме тупикового) способны полностью перевозить поданные ресурсы, а спиттеры работают мгновенно – то сплиттер, подключённый к тупиковой ветке просто уменьшает период своей циклической группы на 1:

$$C[t_i] \xrightarrow{\text{deadend}} C[t_i - 1]$$

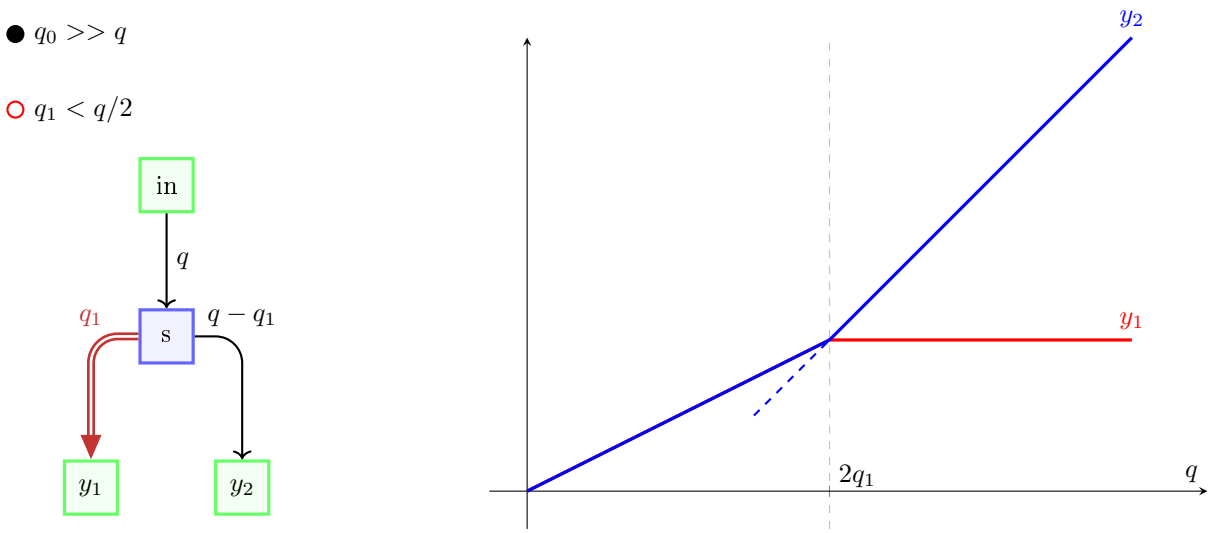
Таким образом тупик можно интерпретировать, как конвейерный разделитель с нулевым количеством выходов, а следовательно, он – пустая циклическая группа $C[0]$.

Однако важно понимать, что уменьшается период лишь одного сплиттера, а не всей системы целиком, что отлично видно на второй схеме.

Благодоря тупиковым ветвям, можно получить преобразование: $\mathbb{T} \to \{2, 3 \ldots, \max(\mathbb{T})\}$, что упрощает построение многих схем.

4.2 Избыток на выход

Рассмотрим теперь вторую ситуацию: $q^{\text{res}} > q_i$:



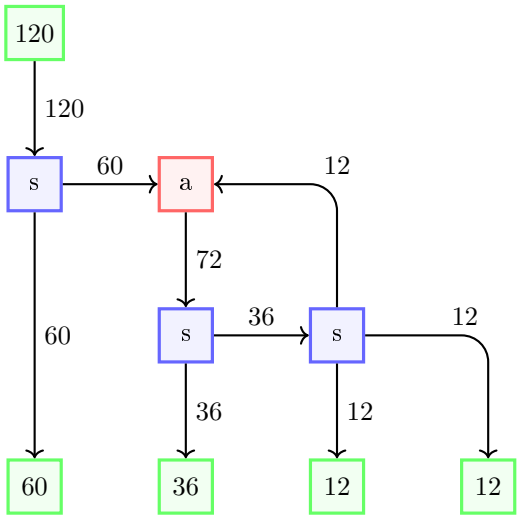
Функции для левого и правого выхода можно написать в общем виде:

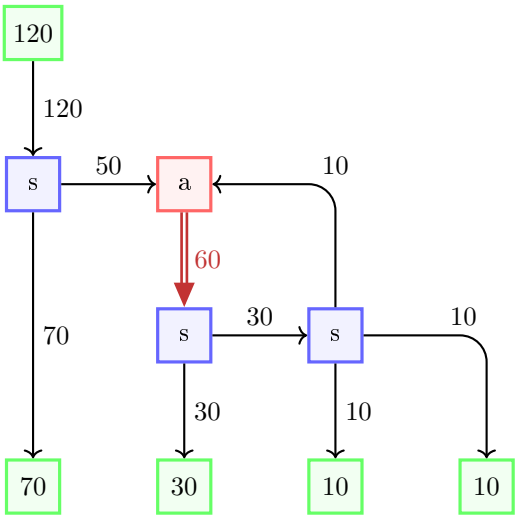
$$y_1(x) = -\frac{|x - 2q_0|}{4} + \frac{x}{4} + \frac{q_0}{2}$$

$$y_2(x) = \frac{|x - 2q_0|}{4} + \frac{3x}{4} - \frac{q_0}{2}$$

Работает данная схема так: если $q_1 \geq q/2$ – то разделитель работает по обычным правилам. В противном же случае появляется новое правило: разделитель первый цикл работает как $C[t_i]$ вплоть до посещения "критического" выхода. Как только до него доходят – запускается таймер на $1/q_1$, и пока он не истечёт – считается, что на этом выходе – тупик - то есть $C[t_i] \to C[t_i - 1]$

Иллюстрация того, **насколько** сильно может измениться суть схемы, если поставить другой конвейер:





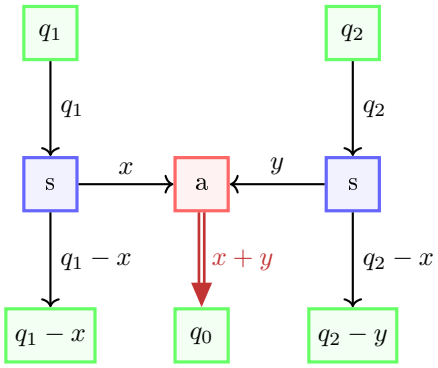
Как видим, такая разница происходит из-за того, что ограниченный конвейер подключён на выход соединителя. Он превращается в тупик по тем же принципам, что и у сплиттера.

Однако здесь у нас 2 **взаимосвязанных** конвейера заходят в "бутылочное горлышко", так что скорость основного из них стабилизируется, как единственное решение уравнения

$$q_0 \cdot \left(1 - \frac{1}{6}\right) = 60$$
$$q_0 \cdot \frac{6}{5} = 60$$
$$q_0 = 50$$

Но тут у нас **единственное** решение – а если их несколько? рассмотрим пример, в котором у нас несколько **независимых** входов:

- $q >> q_1, q_2$
- $q_0 < q_1/2 + q_2/2$

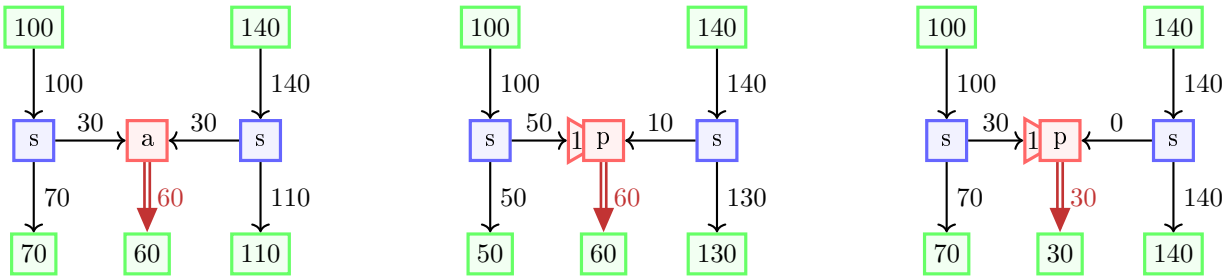


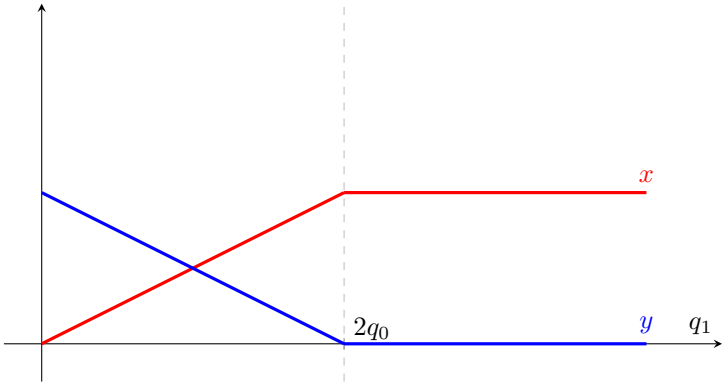
Очень сильно хочется сказать "пускай $x = y$" – тогда $x = \frac{q_0}{2}$. Но на деле нам ничто не гарантирует такого: на деле соединитель может приоритезировать один из входов. Второй случай более интересный: например, в

Satisfactory есть отдельный тип соединителя – приоритетный соединитель (англ: priority merger), в котором есть переключатели степени приоритетности того или иного входа.

Пускай левый вход имеет бóльший приоритет. Рассмотрим возможные ситуации:

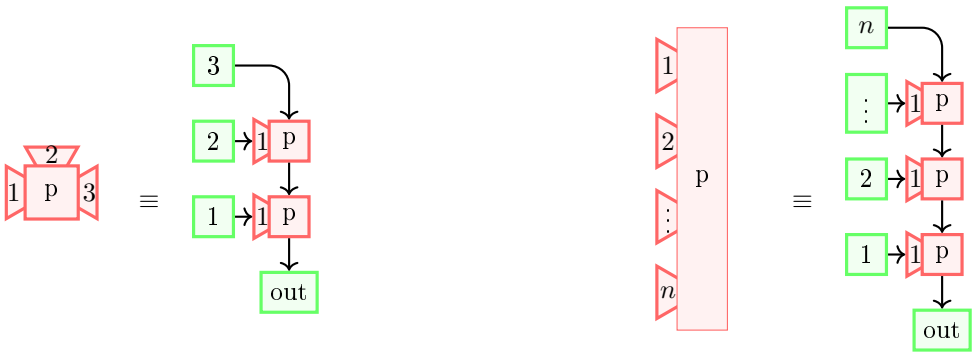
- $\frac{q_1}{2} \le q_0$: $x = \frac{q_1}{2}$, $y = q_0 - \frac{q_1}{2}$
- $\frac{q_1}{2} \ge q_0$: $x = q_0$, $y = 0$





Таким образом приоритетный соединитель работает по следующему алгоритму: когда во вход приходит первый ресурс– он сразу выходит на конвейер q. После этого запускается таймер на $1/q$, на протяжении которого никакие ресурсы не выходят, а просто выстраиваютсяв приоритетную очередь. Когда таймер кончается – ресурс с бóльшим приоритетом выходит, и заново запускается таймер.

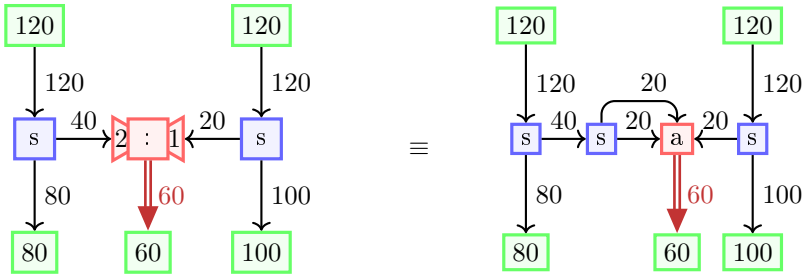
По схожему принципу можно реализовать и другие порядки приоритета, если входов больше двух:

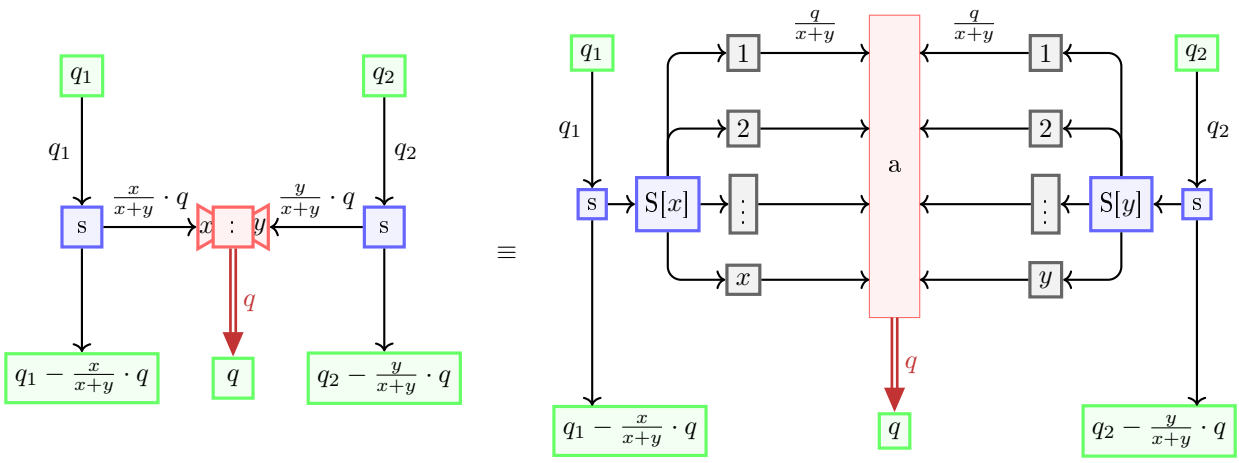


При этом так же можно добавить приоритетный соединитель, который пускает ресурсы только в рамках

заданного соотношения $x : y$ по следующему принципу: без ограничений общности положим, что самый первый ресурс зашёл во вход x. Тогда в этом входе запускается таймер на $\frac{x}{x+y} \cdot \frac{1}{q}$, а на втором, соответственно – $\frac{y}{x+y} \cdot \frac{1}{q}$, на время действия которого данный вход превращается в тупик.

Данную схему также можно реализовать при помощи более простых видов соединителей:



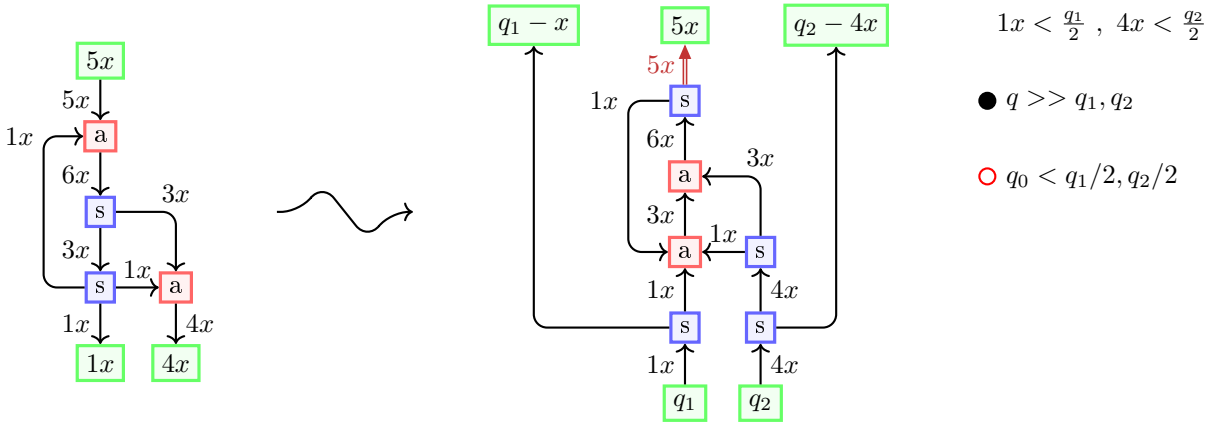


4.3 Теорема об инверсии

Однако интересно, а что если соединитель не может принять неограниченное количество конвейеров? В реальности как раз так и происходит: в Satisfactory множество числа входов $\mathbb{A} = \{2, 3\}$.

Теорема: схема для соединителя $(x : y)$ строится, как инверсия отделителя $(x : y)$:

- Направление конвейеров реверсируется
- Соединители заменяются на разъединители
- Разъединители заменяются на соединители
- Выходная лента имеет строго ограниченную скорость: $(q < \frac{q_1}{2}) \wedge (q < \frac{q_2}{2})$



Заметим, что на выходе из последнего разъединителя почему-то не равное число ресурсов, притом на "быструю" ленту уходит много меньше, чем на "медленную", хотя должно быть наоборот. Это происходит из-за того, что число ресурсов на входе конечного делителя определяется рекуррентным соотношением:

$$g(x) = \frac{1}{4}|x - 2q_0| + \frac{3}{4}x - \frac{q_0}{2}$$
$$f_n(x) = g(f_{n-1}(x)) + x$$
$$h(x) = \lim_{n\to\infty} f_n(x)$$

Где:

- $g(x)$ – функция "сброса" на левом выходе делителя, от поданного на неё числа ресурсов x
- $f_n(x)$ – итерационное значение скорости ресурсов на предпоследней ленте
- x (в параметрах f и h) – то число ресурсов, что заходит от входов

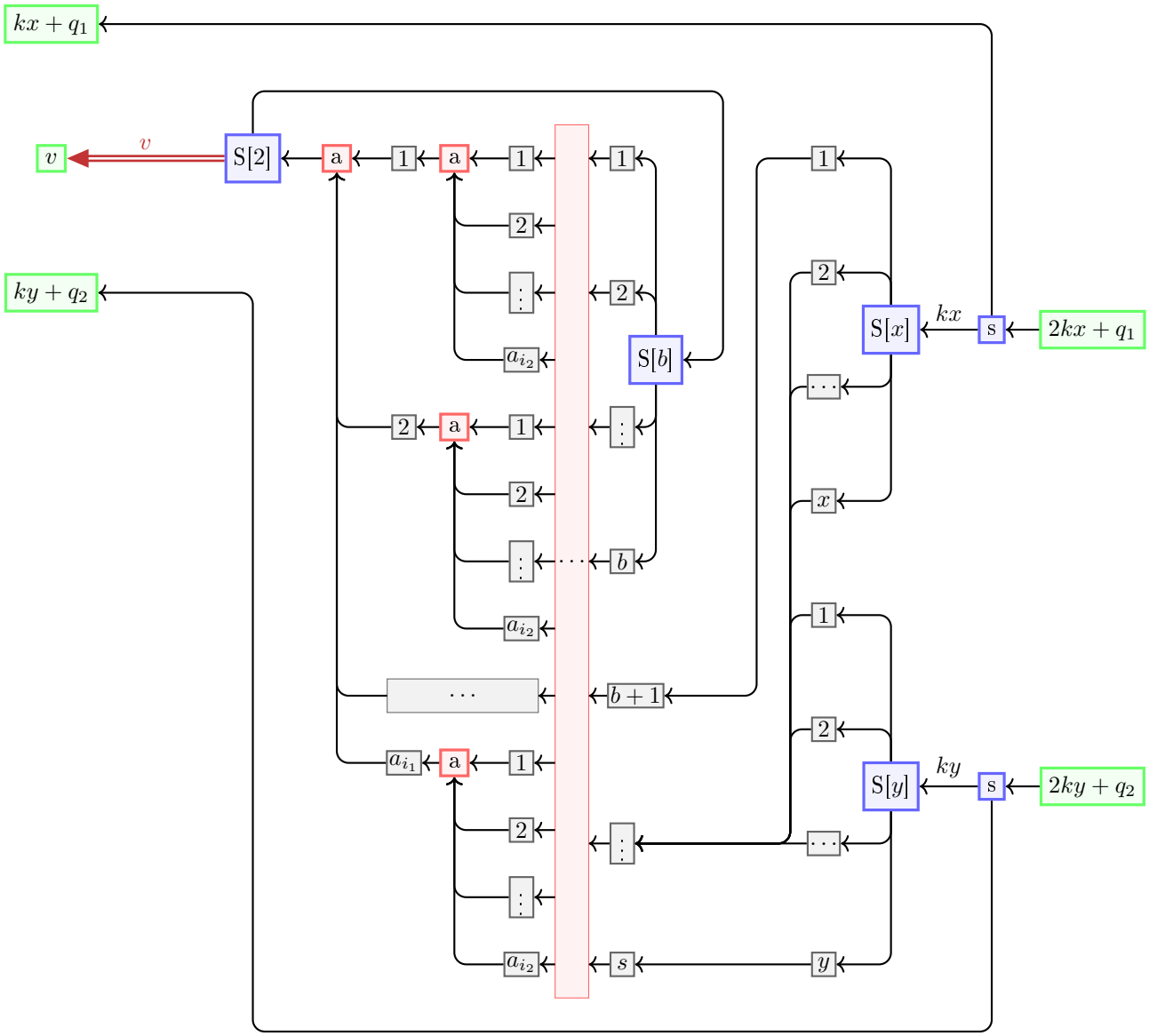
Если положить $x = \frac{q_1}{2} + \frac{q_2}{2}$, как и хочется сделать – то h – расходится, и улетает в бесконечность. Очевидно, такого быть не может, и, даже если $q >> q_1, q_2$ – то это вовсе не значит, что $q = \infty$, поскольку это возможно лишь при $\rho = \infty$, но и это очевидно неверно. Из-за этого, на всех конвейерах внутри схемы происходит затор, они полностью забиваются ресурсами, и останавливаются – превращаются в тупики, и ждут, когда появится новое место на выходе (закончится таймер на главном выходе последнего разъединителя) – и только потом, одна из двух веток на мгновение приходит в движение. При этом, в части случаев, ресурс, который так долго ждал своего выхода – выходит на левый выход, и отправляется обратно.

Если положить $x = 2q_0$ (предельное значение) – то это будет неустойчивое равновесие: $h(2q_0) = 2q_0$, но если по какой-либо причине на вход систематически начнёт поступать хоть немного больше – то очень быстро там снова поедет бесконечное число ресурсов, и механизм встанет.

Если схема построена на основе степенного дерева – то при подаче предельного значения в схеме та ветвь, что соответствует $\max(x, y)$ – будет идти без простоев, а та, что $\min(x, y)$ – встанет (как было описано выше).

Вообще, как можем заметить, в такой схеме сохраняются даже пропускные способности от оригинала. При этом разъединители перестают в обязательном порядке делить поровну, поскольку они становятся образом соединителей, которые могут принимать конвейеры в любых соотношениях.

Нетрудно понять, что если $\mathbb{A} \not\supset \mathbb{T}$ – то такое преобразование в прямом виде – невозможно. Но на основе этого принципа можно построить искомую схему. Обозначим: $s(x+y) = s$, $s-(x+y) = b$, $k\cdot(x+y)$. Реализуем схему для соединения $x + y$ конвейеров, а затем реализуем подсхемы для разделения на x, y и b независимо. Так же понадобится промежуточно реализовать схему для разделения на 2 (если $2 \notin \mathbb{T}$):



Казалось бы, что данная схема очень громоздкая, ведь содержит аш 5 различных подсхем... Наделе все 5, включая схему соединителя – можно сократить по ранее упомянутым же принципам "ёлочки" и "палочки". А если для какого-нибудь $\alpha \in \{x, y, b\}$: $\exists\, t \in \mathbb{T} \mid \alpha < t$ – то такую схему можно реализовать даже без возврата, а с уже известными нам тупиками.

Хорошо, схема может и не такая уж и большая, как кажется – но какой всё же в ней смысл, если для каждых q_1 и q_2 можно реализовать свой разделитель? Вот именно, что для каждых q_1 и q_2 нужно будет реализовывать отдельную схему, а тут – одна для всех. Например, если ресурсы перевозятся поездами с некоторой задержкой с разных производств на другие разные производства – то одну эту схему можно использовать для разделения обоих случаев, а не строить две независимые схемы – Нужно лишь в конце тем, или иным методом сделать распределитель на разные заводы (чем мы займёмся в дальнейших главах).

4.4 Затворы

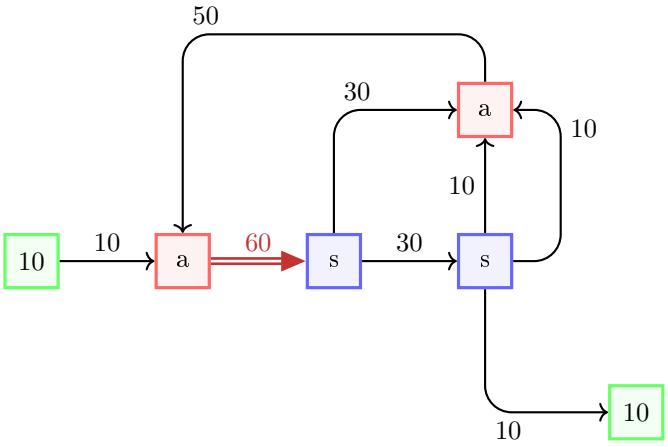
Однако важно учесть, что может быть так, что $k \cdot (x + y) \notin \mathbb{V}$. Обозначив $k \cdot (x + y) = v$. Рассмотрим три возможные ситуации:

Если $v < \min(\mathbb{V})$ – запора происходить не будет, и ресурсы будут просто проскальзывать, и соотношение

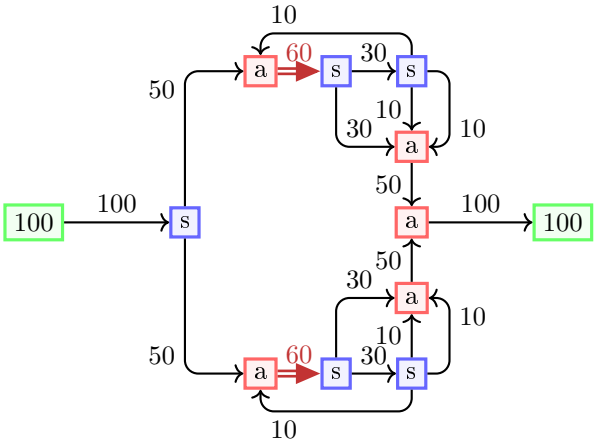
перестанет сохранятся. Можно реализовать затворный механизм: реализуем схему на возврат некоторой доли $\frac{a}{b}$, а оставшиеся конвейеры соединим. Тогда необходимо, чтобы:

$$v \cdot \frac{1}{1-\frac{a}{b}} = \min(\mathbb{V}) \qquad \frac{v}{\min(\mathbb{V})} = 1 - \frac{a}{b} \qquad \frac{a}{b} = \frac{\min(\mathbb{V}) - v}{\min(\mathbb{V})}$$

$$a = \frac{\min(\mathbb{V}) - v}{\gcd(\min(\mathbb{V}), v)} \ , \ b = \frac{\min(\mathbb{V})}{\gcd(\min(\mathbb{V}), v)}$$



Если же $v > \max(\mathbb{V})$ – то наоборот будет происходить перезапор: соотношение будет сохраняться, но числа на выходах будут другие. Воспользуемся принципом чайника и аксиомой Архимеда, и просто разделим входной конвейер на несколько параллельных частей, в каждой из которых $\exists\ u \in \mathbb{V} \mid v < u$, к которому применим затворный механизм:



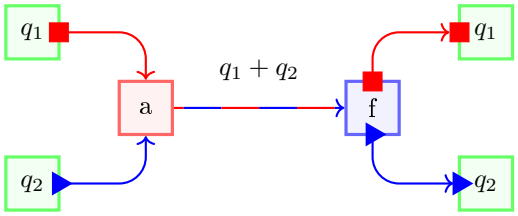
Последняя ситуация: если $\exists\ m, M \in \mathbb{V} \mid m < v < M$ – то в зависимости от конкретных значений v и $\mathbb{V}$ можно использовать оба предыдущих метода.

Chapter 5

Многоресурсные системы

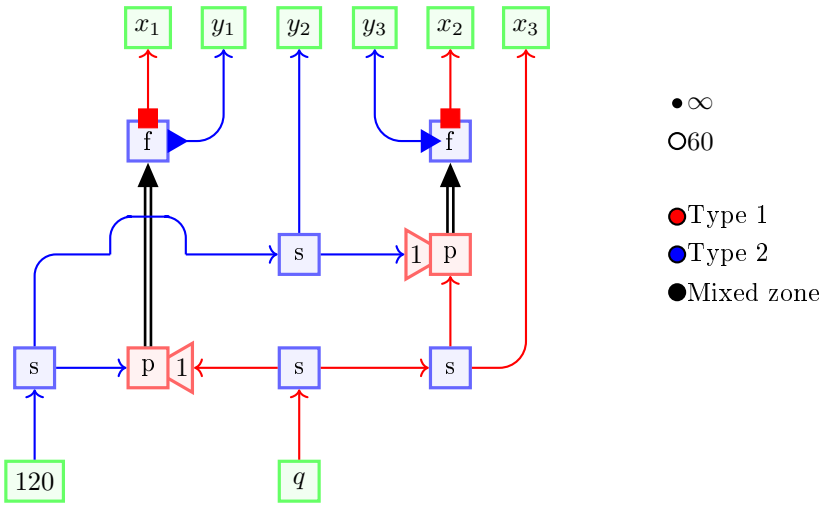
В реальности часто бывает так, что вдоль одной шины идёт транспортировка множества разных типов ресурсов на параллельных конвейерах, притом эти конвейеры полупустые. Для решения этой проблемы часто используют сушибелт (от англ: sushi-belt – линия, конвейер, по которой в некоторых заведениях курсируют суши разных видов). На конце такой ленты используют специальный вид разделителей, который может отфильтровать нужный тип ресурса.

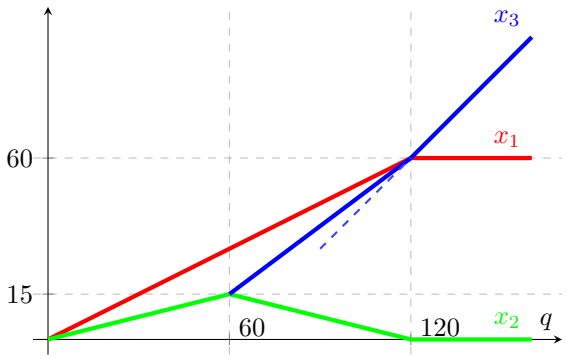
Рассмотрим пример схемы сушибелта для двух ресурсов:

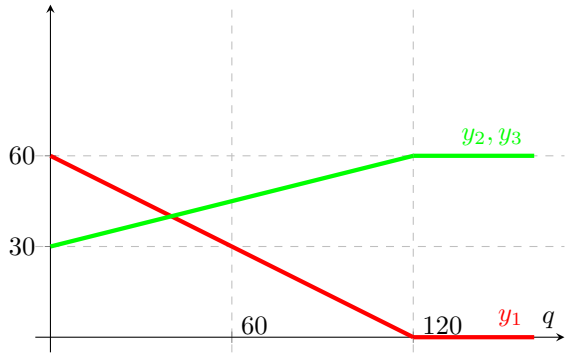


5.1 Вычитатель

Для конвейеров со смешанными ресурсами выполняются все предыдущие теоремы. Но при помощи фильтра можно построить одну интересную схему. Идея этой схемы начинается с того, что мы прекрасно знаем, как "складывать" ресурсы – а можем ли мы их вычитать...? Можем!







Как можем заметить, "вычитание" происходит с двух сторон: по мере роста q, уменьшаются как y_1, так и с какого-то момента и x_2. Если второй тип ресурса закольцевать, и поставить буффер – то получится замкнутая схема, в которой при определённом росте входной скорости, выход на другой – уменьшается.

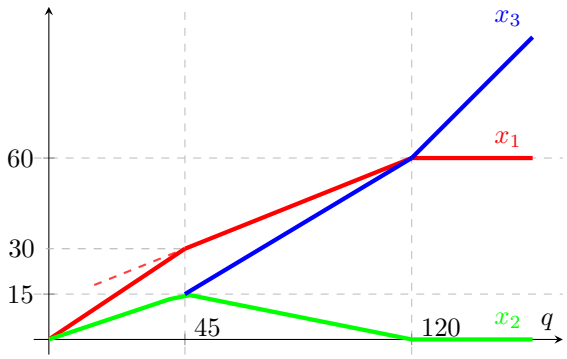
Самый интересный график – $x_2(q)$ – он задаётся функцией:

$$x_2(q) = \frac{1}{8}\left(|q| - 2|q - 60| + |q - 120|\right)$$

Первоначально, чем больше ресурсов идёт на вход – тем больше выход (с коэффициентом 1/4), но после критического значения в $q = 60$ – выход только уменьшается. А что если выход приплюсовать к входу? Подведём на вход критическое значение ресурсов, соответственно, на выходе поедет 15 ресурсов, но когда они приедут на вход – то уменьшат значение на выходе. Но ведь если значение на выходе уменьшилось – то уменьшилось и значение на входе – стало быть, выход вырос. Но раз выход вырос – то вырос и вход – тогда выход уменьшился...

Честно решив рекурсивное уравнение, получим:

$$\begin{aligned}
x_2^0(q) &= \frac{1}{8}\left(|q| - 2|q - 60| + |q - 120|\right) \\
x_2^n(q) &= x_2^0(q + g_{n-1}(q)) \\
x_2 &= \lim_{n\to\infty} x_2^n(x) = \\
&= \frac{1}{30}\left(5|q| - 8|q - 45| + 3|q - 120|\right)
\end{aligned}$$



Интересно, что такое преобразование, по сути, берёт отрезок $[0;\ 60]$, и сдвигает точку 30 в положение 15, сохраняя значения функций в этой точке, и сохраняя непрерывность первоначальной функции. Такое преобразование и описывается сменой оси x на $x + h(x)$, при сохранении первоначальной разметки.

Мы получили однозначный ответ – это большое счастье. На самом деле, многие рекуррентные соотношения такого вида не имеют статический точек на бесконечности – они могут иметь периодический характер решений, а того хуже – апериодический. В общем виде имеем:

$$x_2^0(q) = \frac{1}{8}\left(|q| - 2|q - 60| + |q - 120|\right)$$
$$x_2^n(q) = x_2^0(q + k \cdot g_{n-1}(q)) \ , \ k \in \mathbb{R}^+$$
$$x_2 = \lim_{n \to \infty} x_2^n(x)$$

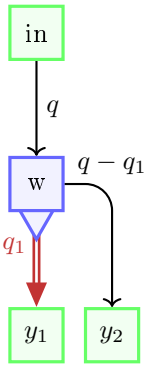
Как можно заметить, для реализации такой системы необходимо реализовать всего лишь один новый элемент – умножитель.

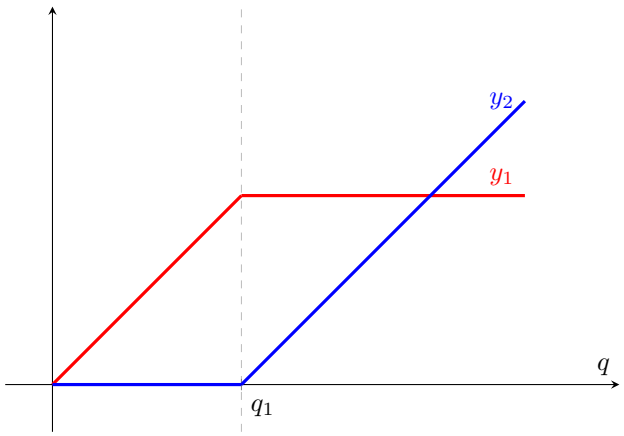
5.2 Вестгейт

Удобнее всего будет реализовать умножитель, используя новый тип делителя – Вестгейт. Данный термин дословно переводится, как "сбрасыватель излишка". Данный объект является логическим продолжением темы, которая была поднята в главе "**Избыток на вход**".

● $q_0 >> q$

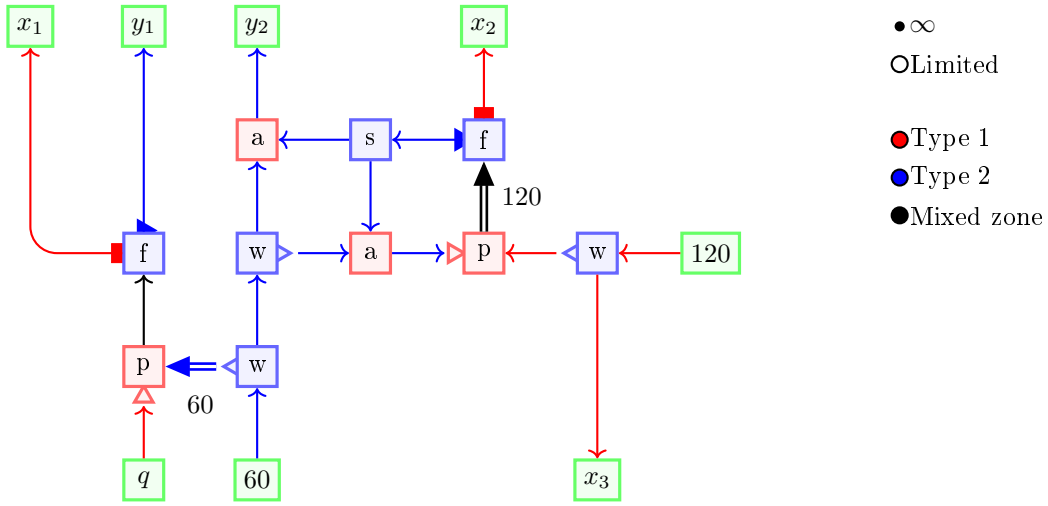
○ $q_1 < q/2$





Этот новый тип делителя позволяет модифицировать все уже известные нам схемы, основанные на принципе излишков таким образом, чтобы сделать их графики более резкими и выраженными. Также, данный делитель позволяет добиться нулевого выхода при ненулевом входе

Собственно, умножитель, например, на 2:


x_1
y_1
y_2
x_2
a
s
f
120
f
w
a
p
w
120
p
w
60
q
60
x_3
∞
Limited
Type 1
Type 2
Mixed zone